INTELLIGENT MULTI-AGENT FRAMEWORK FOR AUTOMATED SOFTWARE TESTING AND BUG ANALYSIS (SENTINELMESH)

COVER PAGE

A PROJECT REPORT

Submitted by

Student Name	Register Number	Department	Official Institutional Email
HARIESH P	**7376241CS195**	Computer Science and Engineering (CSE)	harieshp.cs24@bitsathy.ac.in
MIDHUM M	**7376242AL152**	Artificial Intelligence and Machine Learning (AIML)	midhumm.al24@bitsathy.ac.in

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

Project Guide:
RAMYA R (Guide ID: **CS1906**)
Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)
SATHYAMANGALAM – 638 401, ERODE DISTRICT, TAMIL NADU

ACADEMIC YEAR: 2026–2027

TITLE PAGE

A PROJECT REPORT

Submitted by

Student Name	Register Number	Department	Official Institutional Email
HARIESH P	**7376241CS195**	Computer Science and Engineering (CSE)	harieshp.cs24@bitsathy.ac.in
MIDHUM M	**7376242AL152**	Artificial Intelligence and Machine Learning (AIML)	midhumm.al24@bitsathy.ac.in

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

Project Guide:
RAMYA R (Guide ID: **CS1906**)
Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)
SATHYAMANGALAM – 638 401, ERODE DISTRICT, TAMIL NADU

ACADEMIC YEAR: 2026–2027

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

SATHYAMANGALAM – 638 401

BONAFIDE CERTIFICATE

Certified that this project report titled **"INTELLIGENT MULTI-AGENT FRAMEWORK FOR AUTOMATED SOFTWARE TESTING AND BUG ANALYSIS (SENTINELMESH)"** is the bonafide work of **HARIESH P. (Register No.: 7376241CS195)**, Department of Computer Science and Engineering, and **MIDHUM M. (Register No.: 7376242AL152)**, Department of Artificial Intelligence and Machine Learning, who carried out the project work under my supervision in partial fulfillment for the award of the degree of **BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING** during the academic year 2026–2027.

SUPERVISOR / PROJECT GUIDE
RAMYA R (Guide ID: **CS1906**)
Department of Computer Science & Engineering
Bannari Amman Institute of Technology
Sathyamangalam – 638 401

HEAD OF THE DEPARTMENT
[HOD Name & Designation Placeholder]
Department of Computer Science & Engineering
Bannari Amman Institute of Technology
Sathyamangalam – 638 401

Submitted for the Project Work Viva-Voce examination held on .....

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We, **HARIESH P. (Register No.: 7376241CS195)**, Department of Computer Science and Engineering, and **MIDHUM M. (Register No.: 7376242AL152)**, Department of Artificial Intelligence and Machine Learning, hereby declare that the project report entitled **"INTELLIGENT MULTI-AGENT FRAMEWORK FOR AUTOMATED SOFTWARE TESTING AND BUG ANALYSIS (SENTINELMESH)"** submitted to the Department of Computer Science and Engineering, Bannari Amman Institute of Technology, Sathyamangalam, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering**, is a record of original work carried out by us under the supervision of **RAMYA R (Guide ID: CS1906)**, Department of Computer Science and Engineering.

We further declare that this work has not been submitted previously, in part or in full, for the award of any other degree, diploma, fellowship, or similar title in this or any other university or institution.

Place: Sathyamangalam
Date: [Insert Submission Date]

Signature of Candidate 1
(HARIESH P. - 7376241CS195)

Signature of Candidate 2
(MIDHUM M. - 7376242AL152)

ACKNOWLEDGEMENT

First and foremost, we express our profound gratitude to the Almighty for bestowing the strength, perseverance, and knowledge required to bring this project to fruition.

We express our deepest gratitude to our respected Chairman, Vice Chairman, and Principal of Bannari Amman Institute of Technology for providing an intellectually stimulating environment and state-of-the-art computational infrastructure.

We extend our heartfelt thanks to the Head of the Department, Department of Computer Science and Engineering, for their visionary encouragement, administrative guidance, and consistent academic support throughout this semester.

We register our sincere gratitude and indebtedness to our Project Guide, **RAMYA R (Guide ID: CS1906)**, Department of Computer Science and Engineering, whose constructive critiques, scholarly feedback, and technical mentorship provided the foundation for the architectural design and execution of this work.

We also extend our sincere thanks to all faculty members and technical staff of the Department of Computer Science and Engineering and Department of Artificial Intelligence and Machine Learning for their direct and indirect support during the design and validation phases of this project.

Finally, we convey our love and affectionate thanks to our parents and friends for their continuous patience, motivation, and encouragement throughout the course of our studies.

HARIESH P. (Register No.: 7376241CS195)
Department of Computer Science and Engineering
Email: `harieshp.cs24@bitsathy.ac.in`

MIDHUM M. (Register No.: 7376242AL152)
Department of Artificial Intelligence and Machine Learning
Email: `midhumm.al24@bitsathy.ac.in`

ABSTRACT

Contemporary software engineering pipelines face a fundamental bottleneck: software testing, defect diagnosis, and patch generation remain fragmented, labour-intensive, and computationally fragile. While static analysers flag heuristic code smells and dynamic unit-testing suites expose runtime assertions, developer intervention is invariably required to correlate disparate failure logs, identify root-cause causal chains, and craft syntactically valid regression patches. This project introduces **SentinelMesh**, an autonomous multi-agent architectural framework for end-to-end automated software testing, bug classification, root-cause investigation, and patch verification.

SentinelMesh orchestrates a pipeline of nine autonomous, specialized agents governed by an asynchronous `OrchestratorAgent`. The framework ingests Python source code projects and applies custom Abstract Syntax Tree (AST) parsing to evaluate cyclomatic complexity, code smells (including unparameterized SQL queries, bare exception blocks, division operations, and dynamic code evaluation), and calculate bounded file risk scores. Subsequently, an AST-driven test generation engine synthesizes target `pytest` suites. A sandboxed execution agent executes dynamic regression suites under isolated subprocesses with process-tree resource controls managed via `psutil`, capturing line execution coverage, assertion failures, and test flakiness.

Defects identified across both static and dynamic vectors are unified, deduplicated via MD5 categorical hashing, and passed to a root-cause agent that extracts surrounding AST context slices and constructs deterministic causal failure chains. To address identified bugs, an automated fix agent produces candidate patches and executes pre-flight regression verification within isolated temporary environments. A critic agent performs dual-perspective validation, recording structured debate rounds into an SQLite relational store (configured with Write-Ahead Logging and connection pooling) and deriving a consensus confidence metric. Furthermore, an integrated AST mutation engine injects relational, arithmetic, and boolean mutations to empirically compute the test suite mutation score. A modern React dashboard connects to a FastAPI backend over WebSockets, visualizing real-time agent execution telemetry, risk heatmaps, debate transcripts, and downloadable engineering audit reports. SentinelMesh operates completely offline using rule-based reasoning, while offering extensible adapters for cloud-hosted Large Language Models (LLMs).

Keywords: Multi-Agent Systems, Abstract Syntax Tree, Automated Program Repair, Software Quality Assurance, Sandbox Isolation, Mutation Testing, Static Analysis.

TABLE OF CONTENTS

Section	Title	Page No.
	COVER PAGE	i
	TITLE PAGE	ii
	BONAFIDE CERTIFICATE	iii
	DECLARATION	iv
	ACKNOWLEDGEMENT	v
	ABSTRACT	vi
	LIST OF FIGURES	ix
	LIST OF TABLES	x
	LIST OF ABBREVIATIONS	xi
CHAPTER I	**INTRODUCTION**	**1**
1.1	Background of Software Quality Assurance	1
1.2	The Imperative of Automated Testing in Modern CI/CD	2
1.3	Limitations of Traditional and Manual Defect Analysis	3
1.4	Motivation for Multi-Agent Architectures in Software Engineering	4
1.5	Problem Statement	5
1.6	Aim of the Project	6
1.7	Specific Project Objectives	6
1.8	Scope and Operational Boundaries	7
1.9	Proposed Solution: The SentinelMesh Framework	8
1.10	Significance and Academic Benefits	9
1.11	Organization of the Report	10
CHAPTER II	**LITERATURE SURVEY**	**11**
2.1	Theoretical Foundations of Automated Software Testing	11
2.2	Static Code Analysis and Abstract Syntax Tree (AST) Grammars	12
2.3	Test Case Synthesis and Coverage Metrics	14
2.4	Defect Classification and Automated Root-Cause Analysis	15
2.5	Automated Program Repair (APR) and Patch Verification	17
2.6	Multi-Agent Systems and Collaborative LLM Reasoning	18
2.7	Mutation Testing and Test Suite Adequacy	20
2.8	Comparative Literature Analysis Table	21

Section	Title	Page No.
2.9	Research Gaps Identified	23
CHAPTER III	**OBJECTIVES AND METHODOLOGY**	**25**
3.1	Functional Requirements	25
3.2	Non-Functional Requirements	26
3.3	Hardware and Software Specifications	27
3.4	Existing System Analysis	28
3.5	Proposed SentinelMesh Architecture	29
3.6	End-to-End Execution Workflow	31
3.7	Data Flow Architecture (Level 0 and Level 1 DFD)	33
3.8	Agent Coordination and State Management	35
3.9	Persistence Model and Relational Schema (ER Diagram)	37
3.10	Subprocess Sandboxing and Execution Safety	39
3.11	Error Handling and Fault Tolerance Strategy	40
CHAPTER IV	**SYSTEM DESIGN AND IMPLEMENTATION**	**42**
4.1	Project Structure and Modular Organization	42
4.2	API Gateway, Request Validation, and Lifecycle Management	43
4.3	Multi-Agent Orchestrator (`OrchestratorAgent`)	46
4.4	Static Analysis and AST Risk Scoring Agent	49
4.5	Test Case Generation Agent	52
4.6	Test Execution and Flakiness Detection Agent	54
4.7	Bug Classification and Deduplication Agent	57
4.8	Root-Cause Analysis and Code-Slicing Agent	59
4.9	Auto-Fix and Pre-flight Verification Agent	62
4.10	Critic Reviewer and Agent Debate Consensus Engine	65
4.11	Abstract Syntax Tree Mutation Testing Engine	67
4.12	Report Synthesis and Export Agent	70
4.13	LLM Provider Abstraction Layer (Offline, Gemini, OpenAI)	71
4.14	Persistence Layer and SQLite WAL Optimization	73
4.15	Presentation Layer (React 19, TailwindCSS, WebSockets)	75
CHAPTER V	**TESTING, RESULTS AND DISCUSSION**	**78**
5.1	Experimental Setup and Runtime Environment	78
5.2	Application Startup and Health Check Verification	79

Section	Title	Page No.
5.3	Verification on Reference E-Commerce Benchmark	80
5.4	Static Smell Detection and Risk Scoring Output	82
5.5	Generated Test Suite Execution and Code Coverage	84
5.6	Root-Cause Tracing and Verification of Generated Patches	86
5.7	Multi-Agent Debate and Consensus Verification	88
5.8	Mutation Testing Adequacy Analysis	89
5.9	Verification Status of Automated Test Suites	90
5.10	Qualitative Analysis and Architectural Discussion	92
CHAPTER VI	**CONCLUSION AND FUTURE WORK**	**94**
6.1	Summary of Research and Engineering Achievements	94
6.2	Key Technical Contributions	95
6.3	Verified Implementation Status versus Design Goals	96
6.4	Identified Limitations and Operational Constraints	97
6.5	Future Enhancements	98
	REFERENCES	**100**
	APPENDIX A: PROJECT REPOSITORY STRUCTURE	**104**
	APPENDIX B: REST API ENDPOINTS SPECIFICATION	**107**
	APPENDIX C: DATABASE RELATIONAL SCHEMA (SQL)	**110**
	APPENDIX D: CORE SOURCE CODE IMPLEMENTATION EXCERPTS	**114**
	APPENDIX E: SYSTEM VERIFICATION TEST CATALOGUE	**120**
	APPENDIX F: SETUP AND EXECUTION MANUAL	**123**

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig. 3.1	SentinelMesh High-Level Multi-Tier Architectural Topology	30
Fig. 3.2	End-to-End Orchestration Sequence and Communication Flow	32
Fig. 3.3	Level 0 Context and Level 1 Subsystem Data Flow Diagram (DFD)	34
Fig. 3.4	Multi-Agent Coordination and Cancellation Checkpoint Workflow	36
Fig. 3.5	Relational Entity-Relationship (ER) Database Schema	38
Fig. 4.1	AST Branch-Counting Cyclomatic Complexity Algorithm Topology	50
Fig. 4.2	SandboxRunner Subprocess Isolation and Process-Tree Termination	56
Fig. 4.3	Pre-flight Patch Verification Pipeline inside Temporary Sandbox	63
Fig. 4.4	Dual-Agent Debate and Consensus Computation Mechanism	66
Fig. 4.5	AST Operator Substitution Pipeline in Mutation Engine	68
Fig. 5.1	Verified Health Endpoint Response and Database Status	79
Fig. 5.2	Frontend Real-Time Telemetry and Risk Heatmap Visualization	83
Fig. 5.3	Automated Bug Explorer with Root-Cause and Patch Diff Views	87

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Comparative Analysis of Existing Testing and Defect Analysis Frameworks	21
Table 3.1	Minimum Hardware Specifications for Local Execution	27
Table 3.2	Core Software Stack and Dependency Specifications	27
Table 4.1	Cyclomatic Complexity Branch Types Evaluated via Python AST	50
Table 4.2	Implemented Security and Reliability Heuristic Code Smells	51
Table 4.3	Supported Abstract Syntax Tree Mutation Operator Replacements	69
Table 4.4	Database Table Definitions and Purpose	74
Table 5.1	Verified E-Commerce Benchmark Injected Defects and Agent Outcomes	81
Table 5.2	Test Execution Suite Catalogue and Verification Audit Status	90
Table B.1	SentinelMesh REST API Specification and HTTP Status Codes	107
Table E.1	Detailed Verification Test Suite Execution Matrix	120

LIST OF ABBREVIATIONS

Abbreviation	Expanded Form
API	Application Programming Interface
APR	Automated Program Repair
AST	Abstract Syntax Tree
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DFD	Data Flow Diagram
ER	Entity-Relationship
FIFO	First In, First Out
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
LOC	Lines of Code
MD5	Message Digest Algorithm 5
PID	Process Identifier
QA	Quality Assurance
RAM	Random Access Memory
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
VCS	Version Control System
Vite	Frontend Tooling Build Suite
WAL	Write-Ahead Logging
WS	WebSocket

CHAPTER I

INTRODUCTION

1.1 Background of Software Quality Assurance

In modern computing, software applications govern mission-critical infrastructures spanning healthcare diagnostics, financial clearinghouses, distributed cloud backends, and automotive control systems. As system complexity increases exponentially, the difficulty of guaranteeing software correctness, security, and reliability escalates concurrently. Software Quality Assurance (SQA) comprises the systematic monitoring, evaluation, and enhancement of software engineering artifacts throughout the Software Development Life Cycle (SDLC) [1].

Historically, quality assurance relied on manual exploratory testing, informal code walkthroughs, and rigid inspection checklists. However, modern agile methodologies and rapid deployment cadences—where production releases occur continuously—have rendered purely manual quality governance obsolete. Defects that bypass early verification cycles result in severe technical debt, security breaches, system outages, and astronomical post-release remediation costs. Consequently, formalizing and automating software quality assurance has emerged as one of the most critical endeavors in computer science and software engineering.

1.2 The Imperative of Automated Testing in Modern CI/CD

Continuous Integration and Continuous Deployment (CI/CD) pipelines necessitate automated verification at every code commit. Automated testing enables rapid regression detection, verifying that recent code modifications do not compromise pre-existing functionality. Automated suites typically encompass unit tests, integration tests, contract tests, and end-to-end user journey validations.

Dynamic test runners like `pytest`, paired with code coverage instrumentation, provide empirical indicators of test suite thoroughness. High statement and branch coverage indicate that a substantial fraction of the syntactic execution paths have been exercised. Furthermore, automated testing provides immediate deterministic feedback to developers, reducing the debugging feedback loop from days to minutes.

Despite the widespread adoption of CI/CD frameworks, writing comprehensive test suites remains an arduous task. Developers frequently produce happy-path unit tests while failing to anticipate boundary conditions, off-by-one errors, null dereferences, or unhandled exceptions under pathological inputs. This test writing gap leaves latent defects undetected until production deployment.

1.3 Limitations of Traditional and Manual Defect Analysis

When a test failure occurs or an unexpected runtime exception is triggered, developers face a multi-stage cognitive challenge:

1. **Log Analysis:** Sifting through voluminous stack traces to isolate the offending instruction.
2. **Defect Categorization:** Classifying the failure (e.g., semantic error, security flaw, resource leak, or race condition).
3. **Root-Cause Investigation:** Tracing the backward dynamic dependency chain from the point of failure to the original syntactic defect.
4. **Patch Synthesis:** Writing a targeted remediation that rectifies the root defect without introducing regression failures.
5. **Regression Verification:** Re-executing all unit and integration tests to confirm the validity of the patch.

Traditional static analysis tools (e.g., linters and heuristic rule checkers) flag potential code smells and cyclomatic complexity hotspots. However, they lack runtime semantic awareness and frequently inundate engineers with false-positive alerts. Conversely, dynamic test runners identify failure locations but offer no explanation regarding the fundamental root cause, leaving diagnosis and remediation entirely to human intuition.

1.4 Motivation for Multi-Agent Architectures in Software Engineering

Recent advances in generative artificial intelligence and Large Language Models (LLMs) have opened new horizons for automated code reasoning, synthesis, and repair. However, relying on monolithic, single-prompt LLM invocations introduces significant fragility:

- Single LLM prompts attempt to parse files, infer bugs, predict fixes, and judge patch correctness simultaneously, often leading to hallucinations, hallucinated function signatures, and unverified code changes.
- Large context windows become polluted with irrelevant log entries, obscuring the primary root cause.
- Monolithic systems lack deterministic execution boundaries; an LLM cannot verify whether its generated code actually runs, satisfies syntax rules, or passes unit tests without dynamic execution feedback.

Multi-Agent Systems (MAS) resolve these limitations by decomposing a monolithic task into discrete, specialized agents operating within a structured pipeline [2]. Each agent possesses an explicit, bounded mandate: one agent performs deterministic Abstract Syntax Tree (AST) parsing, another synthesizes pytest functions, a third manages isolated subprocess execution, a fourth isolates root causes, a fifth synthesizes patches, and a sixth acts as an adversarial critic. Through collaborative peer interaction and dynamic execution sandboxes, multi-agent frameworks combine the deterministic guarantees of traditional compiler parsing with the flexible semantic reasoning of modern AI models.

1.5 Problem Statement

Software development teams face high defect remediation latencies, incomplete unit test coverage, and human error during root-cause diagnosis and regression patching. Existing automated testing tools operate in fragmented silos: static linters operate in isolation from test runners; test runners fail without automated causal explanation; and automated program repair tools lack pre-flight regression verification and multi-agent peer review.

There is a critical need for an integrated, autonomous software quality framework that:

1. Performs deterministic static analysis and AST-based complexity profiling.
2. Synthesizes relevant automated test cases tailored to the analyzed source code.
3. Executes test suites in a secured sandbox environment, monitoring execution metrics and flakiness.
4. Categorizes and deduplicates static and dynamic defects.
5. Isolates root-cause causal chains using code-slicing techniques.
6. Automatically generates candidate patches and validates them against regression suites.
7. Conducts adversarial peer review before presenting verified findings to the developer.

1.6 Aim of the Project

The primary aim of this project is to design, develop, and evaluate **SentinelMesh**, an intelligent multi-agent framework that automates the end-to-end lifecycle of software quality assurance, defect diagnosis, root-cause analysis, and self-healing patch verification for Python codebases.

1.7 Specific Project Objectives

To realize the overall aim, the project is structured around the following specific technical objectives:

1. **Architect a Multi-Agent Mesh:** Design an asynchronous orchestration engine coordinating nine specialized agents across static analysis, test generation, test execution, bug classification, root-cause analysis, auto-fixing, critic review, mutation testing, and reporting.
2. **Implement AST-Based Static Analysis:** Develop an AST parsing engine to calculate cyclomatic complexity via branch counting, detect common security and reliability code smells, and assign file risk scores.

3. **Automate Test Suite Synthesis:** Create an automated test generation agent that parses class and function signatures to synthesize executable `pytest` test suites.
4. **Build a Sandboxed Execution Engine:** Implement an isolated subprocess runner with `psutil` process-tree controls to execute test suites safely, compute statement coverage, and flag flaky tests.
5. **Establish Causal Root-Cause Slicing:** Construct an engine that extracts local AST code slices around failure lines and constructs step-by-step causal fault chains.
6. **Implement Pre-Flight Patch Verification:** Develop an automated repair agent that synthesizes code patches and verifies them in a temporary sandbox to confirm regression safety before human review.
7. **Incorporate Adversarial Critic Review:** Implement a dual-perspective critic agent to conduct automated debate rounds and compute an objective consensus confidence score.
8. **Integrate Mutation Testing:** Build an AST mutation engine that replaces relational, arithmetic, and boolean operators to evaluate the empirical adequacy of the generated test suites.
9. **Deliver a Real-Time Web Interface:** Develop a reactive dashboard using React and WebSockets to stream agent logs, risk heatmaps, debate transcripts, and downloadable markdown audit reports.

1.8 Scope and Operational Boundaries

The scope of the SentinelMesh framework is defined as follows:

- **Target Language:** The framework is specifically developed and optimized for Python codebases (`.py` source files and packages).
- **Execution Environment:** Supports local execution on standard computing hardware (Windows, Linux, macOS) without requiring mandatory cloud dependencies.
- **LLM Decoupling:** Implements a fully operational, deterministic `OfflineLLMProvider` using rule-based reasoning, ensuring complete offline functionality without API keys. Pluggable adapters for Google Gemini and OpenAI are included for extended inference.
- **Sandboxing Boundary:** Subprocess isolation with timeout guards and process-tree termination. Full operating-system virtualization (e.g., Docker or hypervisors) is identified as a future production enhancement.
- **Verification Guarantee:** Candidate patches are considered verified only if they resolve the targeted defect and pass the entire project test suite in an isolated temporary directory.

1.9 Proposed Solution: The SentinelMesh Framework

SentinelMesh addresses the shortcomings of existing fragmented testing tools by providing a unified, multi-agent platform. When a developer submits a Python project (via directory path or ZIP upload), SentinelMesh initiates an asynchronous analysis pipeline managed by the `OrchestratorAgent` .

The workflow progresses through nine sequential stages:

1. `StaticAnalysisAgent` inspects the codebase using Python's `ast` module, mapping cyclomatic complexity and detecting security/reliability code smells.
2. `TestGenerationAgent` synthesizes modular `pytest` test suites for identified classes and functions.
3. `TestExecutionAgent` invokes the tests in a temporary workspace, computing pass/fail rates, code coverage, and flakiness.
4. `BugClassificationAgent` aggregates dynamic test failures and static smells, deduplicating them using MD5 categorical hashing.
5. `RootCauseAgent` extracts local code slices and synthesizes causal failure chains.
6. `AutoFixAgent` constructs candidate code patches and verifies them inside an isolated temporary sandbox.

7. `CriticAgent` executes automated debate rounds to establish a consensus confidence score.
8. `MutationEngine` injects AST mutations (operator swaps) to determine test suite mutation kill rates.
9. `ReportAgent` consolidates all metrics into an executive summary and a detailed markdown audit report.

All run metadata, defect records, logs, and debate transcripts are persisted in an ACID-compliant SQLite database using Write-Ahead Logging (WAL) and streamed to the React dashboard via WebSockets.

1.10 Significance and Academic Benefits

The academic and practical significance of this project includes:

- **Architectural Contribution:** Demonstrates how multi-agent coordination bridges the gap between deterministic static analysis and semantic generative repair.
- **Empirical Test Adequacy:** Combines traditional line coverage with AST mutation testing to provide an empirical assessment of test suite quality.
- **Zero-Trust Patch Verification:** Prevents hallucinated or broken code patches from reaching developers by enforcing pre-flight test execution inside an isolated sandbox.
- **Accessibility and Privacy:** Operates entirely offline with zero third-party API dependencies, preserving intellectual property and code confidentiality.

1.11 Organization of the Report

This report is structured into six comprehensive chapters:

- **Chapter I — Introduction:** Presents the background, motivation, problem statement, objectives, scope, and proposed solution.
- **Chapter II — Literature Survey:** Reviews existing academic literature on automated testing, static analysis, AST grammars, automated program repair, and multi-agent systems, concluding with a comparative analysis and research gap identification.
- **Chapter III — Objectives and Methodology:** Details system requirements, architectural topology, data flow diagrams, agent coordination, database design, and sandboxing safety.
- **Chapter IV — System Implementation and Modules:** Provides an in-depth technical analysis of each implemented module, supported by verified file paths, class designs, and source code excerpts.
- **Chapter V — Results and Discussion:** Presents verified experimental findings, test execution matrices, bug detection outputs, patch verification results, and architectural discussions.
- **Chapter VI — Conclusion and Future Work:** Summarizes project outcomes, contributions, verified implementation milestones, and outlines future research directions.
- **Appendices & References:** Includes the project directory layout, REST API documentation, database schema definitions, code excerpts, test catalogues, setup manuals, and IEEE-style bibliography.

CHAPTER II

LITERATURE SURVEY

2.1 Theoretical Foundations of Automated Software Testing

Automated software testing is grounded in the foundational principles of program verification and empirical validation. Dijkstra famously observed that program testing can be used to show the presence of bugs, but never to show their absence [3]. Consequently, software testing research focuses on maximizing fault revelation probability while minimizing computational overhead and human intervention.

Myers et al. [1] formalized testing methodologies into black-box (functional) and white-box (structural) paradigms. In white-box testing, test suites are generated with the explicit objective of traversing internal program logic, characterized by control flow graphs and branch predicates. The primary challenge in automated test synthesis is the state-space explosion problem: as cyclomatic complexity grows, the number of unique execution paths grows exponentially, making exhaustive path testing computationally intractable.

Recent approaches employ search-based software testing (SBST), utilizing genetic algorithms and meta-heuristic search to explore execution paths [4]. While SBST tools such as EvoSuite have achieved high branch coverage in object-oriented environments, they struggle with complex input preconditions, mock synthesis, and human-readable test assertions. This limitation highlights the need for intelligent, context-aware test generation techniques.

2.2 Static Code Analysis and Abstract Syntax Tree (AST) Grammars

Static code analysis examines source code without executing the underlying program, evaluating adherence to syntactic rules, coding standards, and security constraints [5]. At the core of static analysis lies the Abstract Syntax Tree (AST)—a structural, hierarchical representation of source code generated by compiler lexical and syntactic analysis phases [6].

In Python, the built-in `ast` module exposes grammar productions conforming to the formal Python grammar specification [7]. Each code construct—such as function definitions (`FunctionDef`), conditional branches (`If`), iteration loops (`For` , `While`), and binary operations (`BinOp`)—is represented as an immutable tree node.

McCabe [8] introduced Cyclomatic Complexity ($V(G)$) as a fundamental software metric derived from graph theory: $$V(G) = E - N + 2P$$ Where E represents the number of edges, N represents the number of nodes, and P represents the number of connected components in the control flow graph. For an individual routine ($P=1$), cyclomatic complexity simplifies to: $$V(G) = \Pi + 1$$ Where Π denotes the count of decision points (conditional branch predicates).

By traversing an AST, static analysers can calculate cyclomatic complexity without constructing full control-flow graphs. Furthermore, AST node visitors can inspect syntax patterns for anti-patterns and vulnerabilities, such as unescaped SQL string formatting, empty exception handlers, and unsafe dynamic execution (`eval` / `exec`) [9]. While static analysis provides high throughput and exhaustive coverage, it cannot observe runtime data values, necessitating complementary dynamic execution techniques.

2.3 Test Case Synthesis and Coverage Metrics

Automated test case generation aims to synthesize executable assertions that validate program behavior against expected specifications [10]. Traditional approaches rely on symbolic execution, where program inputs are treated as symbolic variables, and constraints are collected along execution branches to be solved by Satisfiability Modulo Theories (SMT) solvers [11]. However, symbolic execution encounters significant difficulties when encountering external API calls, native library invocations, and complex string operations.

Code coverage serves as the primary metric for evaluating test suite thoroughness. Formulated by Miller and Maloney [12], statement coverage measures the fraction of executable source statements exercised during test execution: $$\text{Coverage}_{\text{stmt}} = \left(\frac{\text{Statements Executed}}{\text{Total Executable Statements}} \right) \times 100%$$ While statement coverage provides an initial indicator of test adequacy, high statement coverage alone does not guarantee the absence of faults. Critical boundary bugs frequently reside in unexercised branch combinations or unexpected runtime exceptions under specific parameter types.

Modern dynamic coverage tools, such as `coverage.py`, use Python execution trace functions (`sys.settrace`) to track instruction pointers at runtime. To support continuous integration, automated test generators must produce native `pytest` suites that integrate cleanly with existing coverage measurement tools.

2.4 Defect Classification and Automated Root-Cause Analysis

When automated tests fail, developers must classify the failure and identify its underlying cause. Defect classification categorizes bugs by functional impact and failure symptoms—such as syntax errors, runtime exceptions, type mismatches, division by zero, index errors, and security vulnerabilities [13].

Root-Cause Analysis (RCA) seeks to trace the causal chain backward from the failure point to the exact syntactic defect in the source code. Zeller [14] pioneered delta debugging, an automated technique that isolates failure-inducing inputs by systematically narrowing differences between passing and failing test runs. Agrawal and Horgan [15] introduced dynamic program slicing, which isolates the subset of program statements that actively influenced the values at a specific failure point.

While dynamic slicing is mathematically rigorous, it incurs substantial runtime tracing overhead. In multi-agent architectures, an alternative approach is static context slicing: when an unhandled exception or assertion failure occurs at line L, an agent extracts a localized window ($L - k$ to $L + k$) of the AST, inspects symbol assignments, and constructs a structured causal explanation of the failure mechanism.

2.5 Automated Program Repair (APR) and Patch Verification

Automated Program Repair (APR) focuses on automatically generating valid patches for detected software bugs without human intervention [16]. APR techniques are generally divided into two main categories:

1. **Generate-and-Validate (Search-Based) APR:** Tools such as GenProg [17] apply genetic programming operators (mutation and crossover) to modify the AST, using passing test suites as an objective fitness function.
2. **Semantic (Constraint-Based) APR:** Tools such as SemFix [18] use symbolic execution to extract repair constraints, using SMT solvers to synthesize valid replacement expressions.

A significant challenge in APR is patch overfitting: candidate patches may pass the specific test suite by deleting functionality or hardcoding return values, while failing on unseen valid inputs [19]. To prevent overfitting, modern APR frameworks enforce regression verification: any synthesized patch must be applied to an isolated copy of the workspace and validated against both the failing test and the existing regression suite.

2.6 Multi-Agent Systems and Collaborative LLM Reasoning

The emergence of Large Language Models (LLMs) has opened new avenues for software engineering automation [20]. However, applying a single LLM to end-to-end bug detection and repair often results in hallucinations, invalid syntax, and unverified code changes.

Multi-Agent Systems (MAS) address these limitations by distributing tasks across specialized agents that interact through structured protocols [21]. In software engineering, frameworks like ChatDev [22] and MetaGPT [23] have demonstrated that assigning discrete roles (e.g., Architect, Coder, Reviewer, Tester) improves code generation quality.

In defect analysis, multi-agent collaboration provides distinct benefits:

- **Separation of Concerns:** Distinct agents handle static parsing, test synthesis, execution monitoring, causal analysis, and patch generation independently.
- **Adversarial Review:** A dedicated Critic Agent reviews proposed patches from an opposing perspective, mitigating model overconfidence.
- **Sandboxed Execution Grounding:** Agents receive empirical execution feedback from isolated sandboxes, anchoring their reasoning in concrete runtime realities rather than speculative model weights.

2.7 Mutation Testing and Test Suite Adequacy

While code coverage measures which statements are executed, it does not measure whether the test assertions are capable of detecting faults. Mutation testing provides an empirical measure of test suite fault-detection effectiveness [24].

In mutation testing, syntactic mutants are generated by applying small, systematic modifications to the source code (e.g., swapping `==` with `!=`, or changing `+` to `-`). The test suite is then executed against each mutant:

- If a test fails, the mutant is considered **killed** (the test suite detected the fault).
- If all tests pass, the mutant **survives** (the test suite failed to detect the fault).

The Mutation Score (MS) is defined as: $$MS = \left(\frac{M_{\text{killed}}}{M_{\text{total}} - M_{\text{equivalent}}} \right) \times 100\%$$ Where M_{killed} is the number of killed mutants, M_{total} is the total number of mutants generated, and $M_{\text{equivalent}}$ represents syntactically altered mutants that retain identical semantics to the original program. High mutation scores correlate strongly with real-world fault detection capability, making mutation testing a valuable validation mechanism for automated test suites.

2.8 Comparative Literature Analysis Table

Table 2.1 summarizes the capabilities and limitations of existing testing and defect analysis approaches in comparison with the SentinelMesh framework.

Table 2.1: Comparative Analysis of Existing Testing and Defect Analysis Frameworks

Framework / Methodology	Primary Paradigm	Core Strength	Key Limitation	Relevance / Contrast to SentinelMesh
SonarQube [5]	Static Analysis & Linters	Comprehensive static rule set and technical debt metrics	Zero runtime awareness; cannot verify bugs dynamically	SentinelMesh incorporates AST static analysis but connects findings directly to dynamic test generation.
EvoSuite [4]	Search-Based Test Generation (GA)	High branch coverage on object-oriented code	Produces complex, hard-to-read tests; cannot synthesize patches	SentinelMesh generates clean, human-readable `pytest` suites and provides automated root-cause repair.
GenProg [17]	Search-Based APR (Genetic Prog.)	Language-agnostic patch search via AST transformations	High patch overfitting rate; high computational cost	SentinelMesh uses targeted causal patch synthesis backed by pre-flight regression sandboxing.

Framework / Methodology	Primary Paradigm	Core Strength	Key Limitation	Relevance / Contrast to SentinelMesh
Pynguin [10]	Automated Test Generation for Python	Automated test generation using genetic algorithms	High assertion generation overhead; does not analyze root causes	SentinelMesh generates tests and couples them with causal slicing and patch verification.
MetaGPT / ChatDev [22, 23]	Multi-Agent LLM Framework	Coordinated multi-agent software artifact generation	Lacks isolated execution sandboxes for empirical test/patch validation	SentinelMesh grounds all multi-agent actions in isolated subprocess sandboxes and AST mutation testing.
SentinelMesh (This Work)	Hybrid Multi-Agent Mesh + AST + Sandbox	End-to-end integration: AST analysis, test generation, sandboxed execution, root-cause slicing, verified patching, critic debate, and mutation scoring	Currently scoped to Python codebases; relies on subprocess rather than OS-level virtualization	Provides a unified, verifiable, and offline-capable software quality platform.

2.9 Research Gaps Identified

Based on the literature survey, several key research gaps were identified in existing automated software quality frameworks:

1. **Isolation of Static and Dynamic Tools:** Static linters and dynamic test runners operate as disconnected tools, requiring manual developer effort to correlate code smells with dynamic test failures.
2. **Absence of Pre-Flight Patch Verification:** Most automated repair tools produce candidate code modifications without rigorously verifying that the patch resolves the bug without breaking pre-existing tests.
3. **Overconfidence and Hallucination in AI Repair:** Existing LLM-based coding tools generate fixes without adversarial peer review or empirical runtime validation, frequently introducing syntax errors and regression bugs.
4. **Lack of Integrated Test Adequacy Assessment:** Test generation frameworks rarely evaluate their synthesized suites against mutation testing, relying solely on line coverage metrics that do not measure fault-detection capability.
5. **Cloud and API Dependency:** Most modern AI-driven developer tools require external API connectivity, raising data privacy concerns and rendering them unusable in air-gapped or offline development environments.

SentinelMesh was explicitly designed to address these gaps through a unified, nine-agent architecture grounded in local execution, deterministic AST analysis, and pre-flight regression sandboxing.

CHAPTER III

OBJECTIVES AND METHODOLOGY

3.1 Functional Requirements

The SentinelMesh framework must satisfy the following functional requirements:

1. **FR-01: Multi-Format Project Ingestion:** Ingest Python projects via local file directory paths or compressed ZIP archives, with input sanitization against path traversal (Zip-Slip) attacks.
2. **FR-02: AST Static Analysis and Complexity Profiling:** Parse Python source files into Abstract Syntax Trees, calculate cyclomatic complexity via branch counting, detect security and reliability code smells, and assign file risk levels.
3. **FR-03: Automated Test Suite Generation:** Synthesize valid, runnable `pytest` test suites exercising detected classes and functions.
4. **FR-04: Isolated Test Execution:** Execute test suites within isolated subprocesses, capture stdout/stderr, measure statement coverage via `coverage.py`, and detect flaky tests through multi-run re-execution.
5. **FR-05: Defect Classification and Deduplication:** Ingest failure records from both static analysis and dynamic test runs, deduplicating findings using MD5 categorical hashing.
6. **FR-06: Localized Root-Cause Slicing:** Extract localized code slices around defect lines and construct structured causal explanations.
7. **FR-07: Automated Patch Synthesis and Pre-Flight Verification:** Synthesize candidate code patches and verify them within an isolated temporary sandbox, confirming regression safety before human review.
8. **FR-08: Dual-Agent Critic Debate:** Conduct structured debate rounds between proposer and reviewer agents, recording debate transcripts and calculating consensus confidence scores.
9. **FR-09: AST-Driven Mutation Testing:** Inject relational, arithmetic, and boolean mutations into source files to compute empirical test suite mutation scores.
10. **FR-10: Multi-Format Report Synthesis:** Generate structured executive summaries and technical markdown audit reports, accessible via REST APIs and WebSockets.

3.2 Non-Functional Requirements

The framework is designed to fulfill the following non-functional requirements:

1. **NFR-01: Execution Isolation and Safety:** Test execution and patch verification must run in isolated temporary directories. Subprocesses must be monitored with timeout guards and process-tree termination via `psutil` to prevent runaway processes.
2. **NFR-02: Zero-Dependency Offline Operation:** Core analysis, test synthesis, bug classification, and patch generation must function entirely offline without external API keys via the `OfflineLLMProvider`.
3. **NFR-03: Concurrency and Thread Safety:** Database access must support concurrent operations using SQLite Write-Ahead Logging (WAL) and SQLAlchemy connection pooling.
4. **NFR-04: Real-Time Telemetry:** Analysis progress, agent logs, and state transitions must be streamed to the user interface via WebSockets with sub-second latency.
5. **NFR-05: Fault Tolerance and Graceful Degradation:** Individual agent errors must be handled gracefully with fallback defaults, ensuring downstream agents continue execution without crashing the entire run.
6. **NFR-06: Usability and Ergonomics:** The web dashboard must provide clear visualizations, risk heatmaps, side-by-side patch diffs, and downloadable markdown reports.

3.3 Hardware and Software Specifications

Table 3.1: Minimum Hardware Specifications

Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (Dual-Core, 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 or higher (Quad-Core+)
System Memory (RAM)	4 GB DDR4	8 GB DDR4 / DDR5 or higher
Storage Space	2 GB available disk space (SSD preferred)	10 GB available SSD space
Network Interface	Local loopback interface (`127.0.0.1`)	High-speed Internet (optional, for cloud LLM APIs)

Table 3.2: Core Software Stack and Dependency Specifications

Layer / Role	Technology / Package	Verified Version in Project
Runtime Environment	Python	3.11 / 3.12 (Standard CPython)
Web Framework & API	FastAPI	$\ge 0.110.0$
ASGI Web Server	Uvicorn (standard)	$\ge 0.28.0$
Data Validation	Pydantic	$\ge 2.6.0$
Database & ORM	SQLAlchemy / SQLite	$\ge 2.0.0$
WebSocket Engine	WebSockets	≥ 12.0
Test Runner & Coverage	pytest, pytest-cov, coverage	pytest ≥ 8.0, coverage ≥ 7.4
System & Process Control	psutil, aiofiles	psutil ≥ 5.9, aiofiles ≥ 23.2
HTTP Client	HTTPX	$\ge 0.27.0$
Frontend Framework	React / Vite	React 19, Vite 6
UI Styling	TailwindCSS	v4.0

3.4 Existing System Analysis

In standard software engineering workflows, quality assurance relies on fragmented, disconnected tools:

- **Linters (e.g., Flake8, Ruff):** Check syntax and style conventions, but lack runtime awareness and produce high volumes of non-actionable alerts.
- **Test Runners (e.g., Pytest):** Execute manually written assertions. When tests fail, they provide raw stack traces but offer no root-cause explanations or repair suggestions.
- **Manual Debugging:** Developers manually analyze logs, formulate hypotheses, edit source code, and re-run tests—a slow and error-prone process.
- **Single-Prompt LLM Assistants:** Developers copy stack traces into chat interfaces. These models frequently hallucinate invalid code and cannot verify that their proposed fixes actually compile or pass regression suites.

The existing paradigm suffers from high diagnostic latency, lack of regression verification, and vulnerability to human oversight during patch development.

3.5 Proposed SentinelMesh Architecture

SentinelMesh replaces this fragmented workflow with a unified, multi-agent architecture. The system is organized into four core layers:

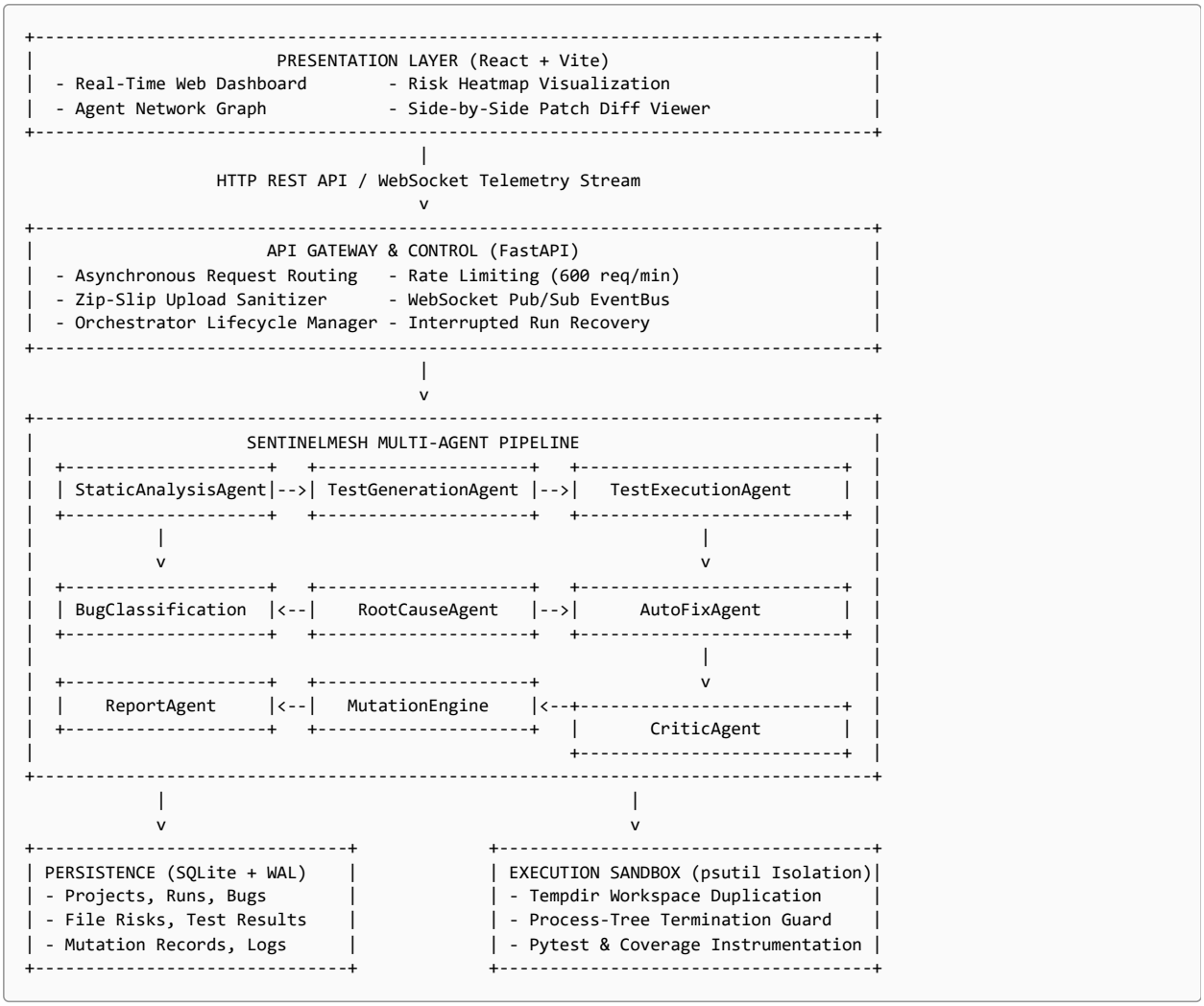


Fig. 3.1: SentinelMesh High-Level Multi-Tier Architectural Topology

1. **Presentation Layer:** A responsive web application built with React 19, Vite, and TailwindCSS. It provides real-time agent execution telemetry, risk heatmaps, bug inspection panels, side-by-side patch diff views, and downloadable audit reports.
2. **API Gateway and Control Layer:** Implemented in FastAPI, providing RESTful endpoints and WebSocket channels. Includes request validation, rate limiting (600 requests/minute), archive upload sanitization, and database connection lifecycle management.
3. **Multi-Agent Pipeline Layer:** Nine specialized agents running asynchronously under the supervision of the `OrchestratorAgent`. Each agent performs a dedicated phase of analysis, testing, diagnosis, or repair.
4. **Execution and Persistence Layer:** An execution boundary utilizing isolated temporary directories and `psutil` process controls, backed by an ACID-compliant SQLite relational database using Write-Ahead Logging (WAL) and SQLAlchemy connection pooling.

3.6 End-to-End Execution Workflow

The execution lifecycle of a SentinelMesh analysis run proceeds through nine sequential stages:

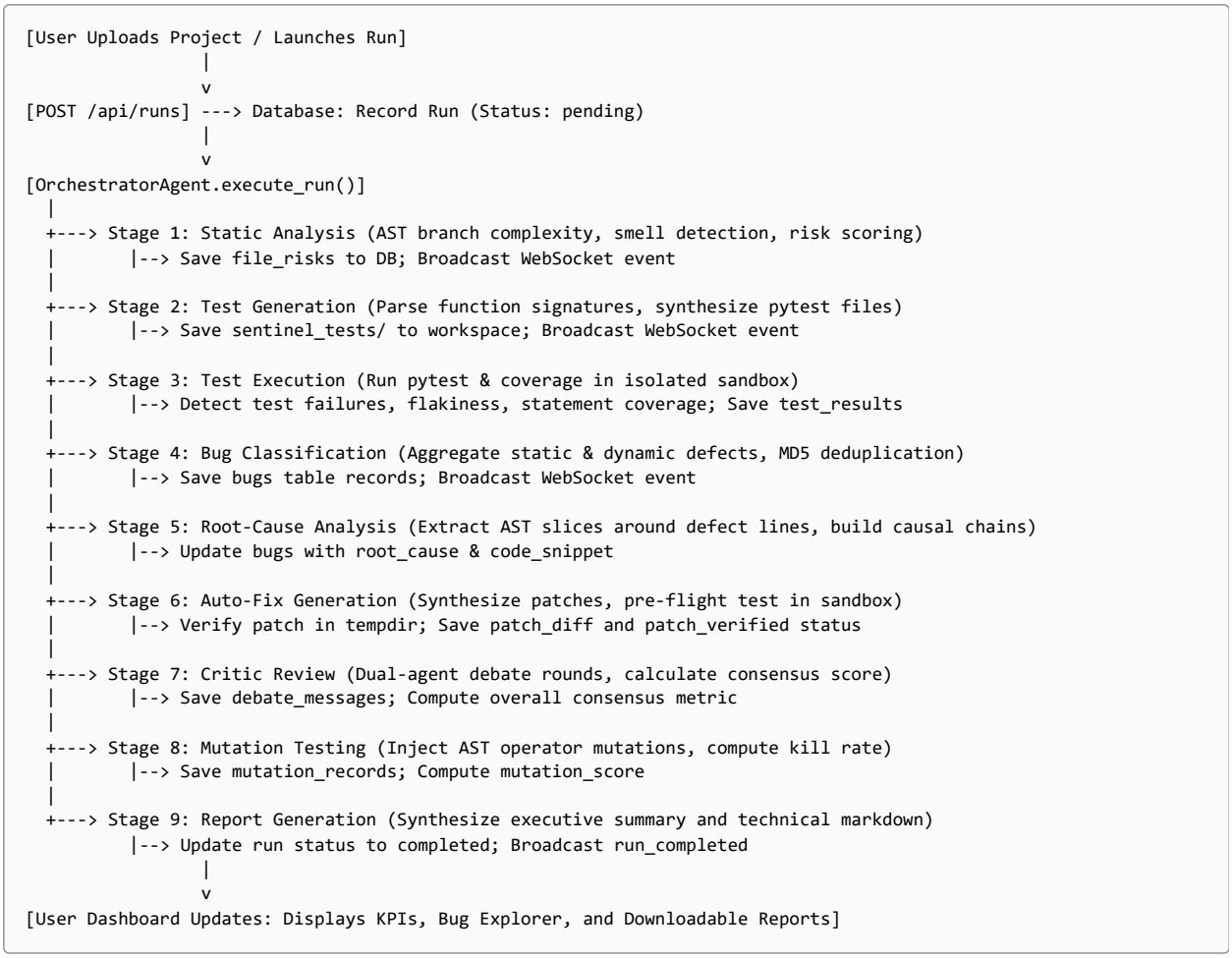


Fig. 3.2: End-to-End Orchestration Sequence and Communication Flow

Between each pipeline stage, the `OrchestratorAgent` checks an explicit cancellation checkpoint (`is_cancelled(run_id)`). If a user cancels the run via the API, the orchestrator terminates downstream execution immediately, ensuring system responsiveness.

3.7 Data Flow Architecture (Level 0 and Level 1 DFD)

The flow of data through SentinelMesh is represented at two levels of abstraction:

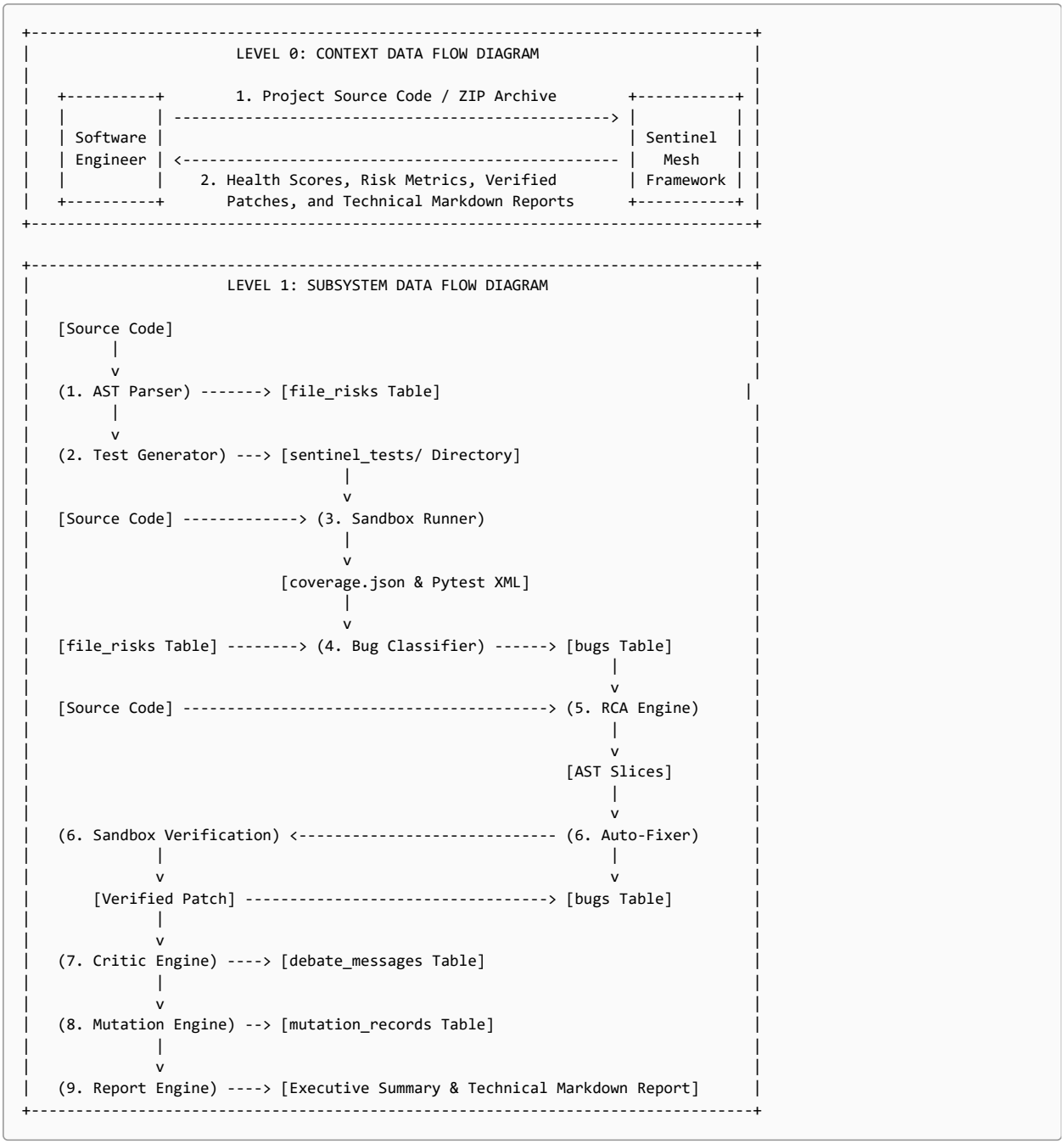


Fig. 3.3: Level 0 Context and Level 1 Subsystem Data Flow Diagram (DFD)

- **Level 0 (Context):** The user provides Python source code and receives health scores, defect classifications, verified patches, and audit reports.
- **Level 1 (Subsystem):** Source code is transformed into AST representations, generating file risk metrics and test suites. Dynamic test execution yields coverage data and failure logs, which are classified and deduplicated into the `bugs` table. The root-cause engine extracts code slices, guiding the auto-fix agent. Synthesized patches are verified in isolated sandboxes before critic review, mutation testing, and final report generation.

3.8 Agent Coordination and State Management

Agent coordination is managed by the `OrchestratorAgent`, which enforces strict execution sequencing, state propagation, and fault recovery:

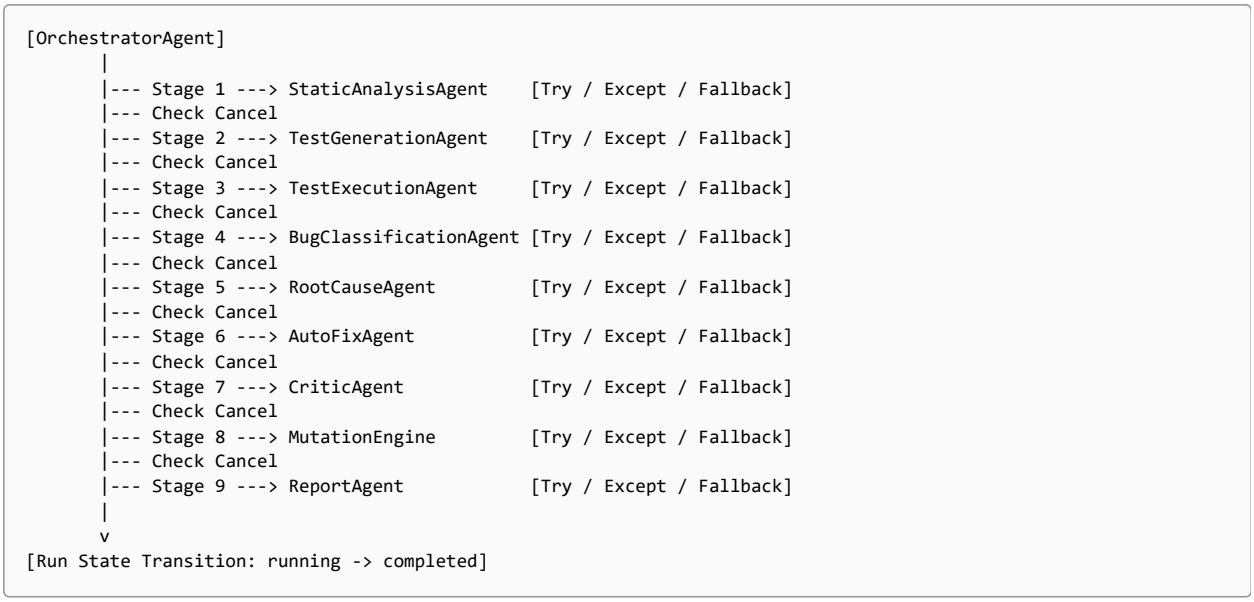


Fig. 3.4: Multi-Agent Coordination and Cancellation Checkpoint Workflow

Key coordination mechanisms include:

- **Sequential Context Propagation:** Each agent reads the outputs of prior stages from the database and writes its own results back to the database, ensuring clean decoupling between agents.
- **Per-Stage Exception Isolation:** Each agent execution is wrapped in a `try-except` block. If an agent encounters an unhandled exception, it logs the error, falls back to safe default values, and allows downstream agents to continue.
- **Database-Backed Cancellation:** The orchestrator queries `is_cancelled(run_id)` between stages, enabling rapid cancellation without leaving orphaned background processes.
- **WebSocket Event Broadcasting:** Every milestone (e.g., `tests_generated`, `tests_executed`, `bugs_detected`) triggers an event on the `EventBus`, which broadcasts the update to connected WebSocket clients.

3.9 Persistence Model and Relational Schema (ER Diagram)

The relational persistence layer is implemented using SQLAlchemy and SQLite. The schema comprises seven core tables:

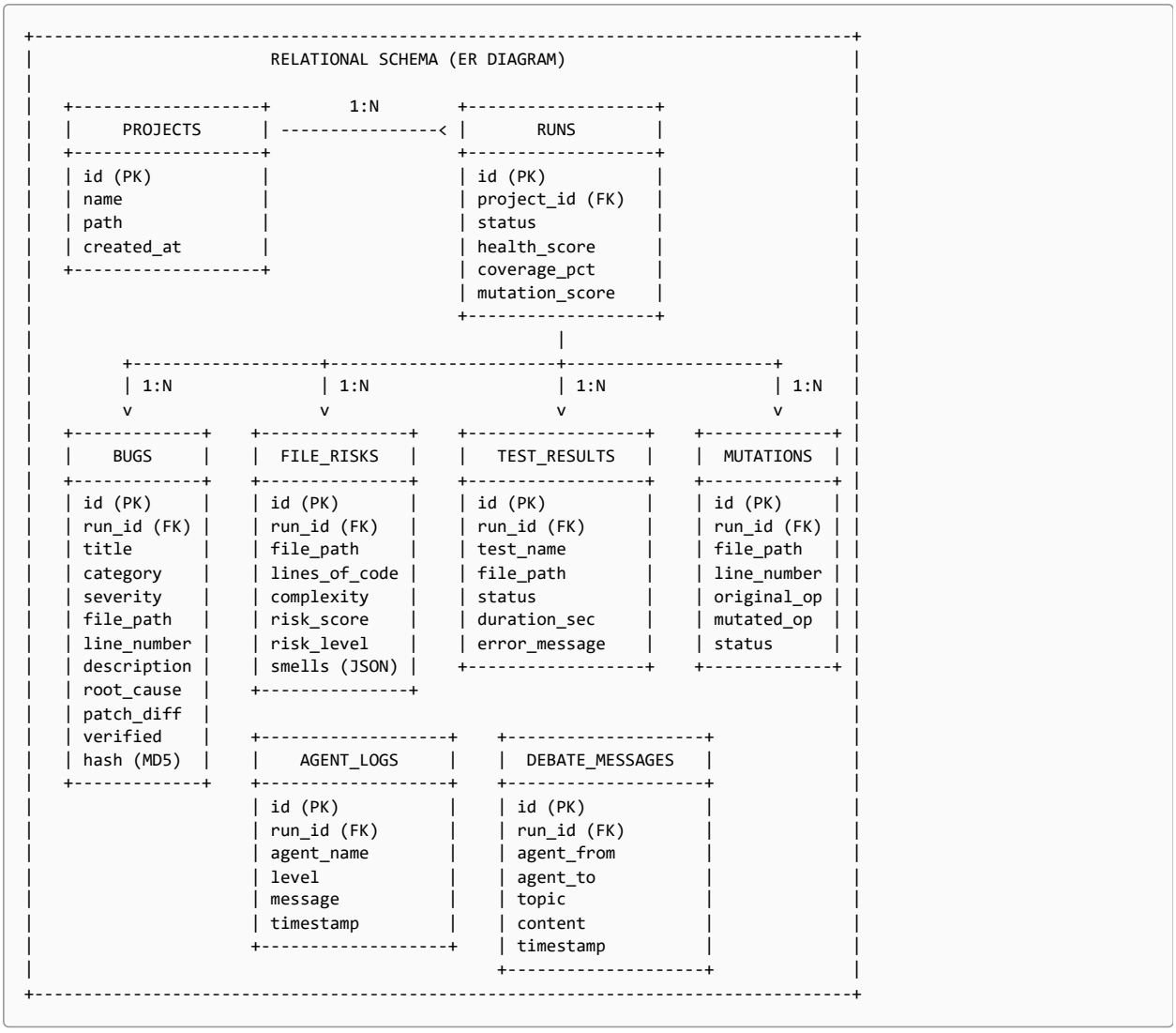


Fig. 3.5: Relational Entity-Relationship (ER) Database Schema

To optimize SQLite for concurrent reads and writes, the database is configured with:

- `PRAGMA journal_mode = WAL` (Write-Ahead Logging for non-blocking concurrent reads).
- `PRAGMA foreign_keys = ON` (Referential integrity enforcement).
- `PRAGMA busy_timeout = 5000` (5-second retry timeout to prevent database lock contention).
- `QueuePool(size=20, max_overflow=10)` (Connection pooling for concurrent API requests).

3.10 Subprocess Sandboxing and Execution Safety

Executing dynamically generated test code and candidate patches introduces inherent security and stability risks. A test script might enter an infinite loop, consume excessive memory, or execute destructive filesystem commands.

SentinelMesh mitigates these risks through a dedicated `SandboxRunner` architecture:

1. **Workspace Duplication:** Dynamic test execution and patch verification are never performed in-place on the user's primary project directory. Instead, `create_isolated_workspace()` duplicates the project into a temporary directory created via `tempfile.mkdtemp()`. Common non-source directories (such as `.git`, `.venv`, and `__pycache__`) are excluded from duplication.

2. **Subprocess Isolation:** Commands are executed as child processes using Python's `asyncio.create_subprocess_exec`, separating the test execution environment from the parent API server process.
3. **Execution Timeouts:** Every subprocess invocation is bound by an explicit execution timeout (default: 60 seconds). Timeouts are enforced using `asyncio.wait_for`.
4. **Process-Tree Termination:** When a timeout occurs or a run is cancelled, standard process termination (`process.kill()`) may leave orphaned child processes running. SentinelMesh uses `psutil.Process(pid).children(recursive=True)` to identify and terminate the entire process hierarchy before terminating the parent process.
5. **Path Traversal Protection:** The archive ingestion engine inspects uploaded ZIP files to verify that all target extract paths resolve strictly within the designated temporary directory, preventing Zip-Slip attacks.

3.11 Error Handling and Fault Tolerance Strategy

To ensure operational stability, SentinelMesh implements a multi-tier fault tolerance strategy:

- **Server Startup Crash Recovery:** When the backend boots, it scans the `runs` table for any runs marked as `running` or `pending`. These represent runs that were interrupted by a prior server shutdown. The server automatically updates their status to `interrupted`, preventing orphaned runs from blocking the system.
- **Agent-Level Fallbacks:** If an agent encounters an unhandled exception during execution:
  - `StaticAnalysisAgent` falls back to empty risk lists with default risk scores of zero.
  - `TestGenerationAgent` logs the error and allows downstream agents to evaluate pre-existing tests.
  - `TestExecutionAgent` falls back to zero-pass metrics with a baseline coverage estimate.
  - `CriticAgent` assigns a default baseline consensus score of 50.0%.
  - `ReportAgent` synthesizes an error-summary report detailing which stages completed successfully.
- **Database Reconnect Resiliency:** SQLite connection timeouts are handled via the 5000ms busy timeout configuration, preventing transient lock contention from raising unhandled database errors.

CHAPTER IV

SYSTEM DESIGN AND IMPLEMENTATION

4.1 Project Structure and Modular Organization

The SentinelMesh codebase is organized into four main packages: `backend`, `agents`, `sandbox`, and `frontend`. The project layout is structured as follows:

```
S5 Mini Project/
├── backend/                 # FastAPI Application Core
│   ├── main.py              # REST API Routes, Middleware, WebSockets
│   ├── database.py          # SQLAlchemy Engine, WAL Configuration, Sessions
│   ├── models.py            # Relational Database Models (SQLAlchemy ORM)
│   ├── schemas.py           # Pydantic Request/Response Data Transfer Objects
│   └── event_bus.py         # Asynchronous In-Memory Event Streaming Bus
├── agents/                  # Autonomous Agent Implementation Package
│   ├── base_agent.py        # Abstract BaseAgent Class and Common Protocols
│   ├── orchestrator_agent.py # End-to-End Pipeline Orchestration Engine
│   ├── static_analysis_agent.py # AST Complexity Profiling and Smell Detection
│   ├── test_generation_agent.py # AST-Driven Pytest Synthesis Engine
│   ├── test_execution_agent.py  # Pytest Runner, Flakiness & Coverage Profiler
│   ├── bug_classification_agent.py # Defect Deduplication & Categorization
│   ├── root_cause_agent.py   # Context Slicing & Causal Chain Analyzer
│   ├── auto_fix_agent.py     # Patch Generator & Sandbox Verification Engine
│   ├── critic_agent.py       # Dual-Agent Debate & Consensus Reviewer
│   ├── report_agent.py       # Technical Markdown & Executive Summary Generator
│   └── llm/                  # LLM Provider Abstraction Layer
│       ├── base.py           # Abstract Base LLM Interface
│       ├── offline_provider.py # Rule-Based Deterministic Offline Provider
│       ├── gemini_provider.py  # Google Gemini API Integration Adapter
│       └── openai_provider.py  # OpenAI API Integration Adapter
├── sandbox/                 # Execution Isolation and Mutation Testing
│   ├── runner.py            # Subprocess Sandbox with psutil Termination
│   └── mutator.py           # AST-Based Operator Substitution Engine
├── frontend/                # Presentation Layer (React 19 + Vite)
│   ├── src/
│   │   ├── App.jsx          # Main Dashboard View and WebSocket Integration
│   │   └── components/      # UI Components (KPICards, RiskHeatmap, etc.)
│   └── package.json         # Frontend Dependency Manifest
├── demo/ecommerce/          # Benchmark Project for Empirical Validation
├── tests/                   # Automated System and Unit Test Suite
├── run.bat / run.sh         # Cross-Platform Execution Scripts
└── requirements.txt         # Python Dependency Manifest
```

4.2 API Gateway, Request Validation, and Lifecycle Management

The API gateway is implemented in `backend/main.py` using FastAPI. It manages request routing, input validation, rate limiting, and database session lifecycles.

Key implementation features include:

- **CORS Configuration:** Allows cross-origin requests from frontend development servers, permitting all HTTP methods and headers.
- **GZip Compression:** Middleware automatically compresses HTTP response payloads exceeding 1000 bytes.
- **In-Memory Rate Limiting:** A custom middleware restricts clients to 600 requests per minute per IP address, preventing resource exhaustion.
- **Startup Crash Recovery:** An application lifespan context manager scans the database at boot, transitioning any stuck `running` or `pending` runs to `interrupted`.

```
# Source File Excerpt: backend/main.py
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: Initialize DB and recover interrupted runs
    init_db()
    db = SessionLocal()
    try:
        stuck_runs = db.query(Run).filter(
            Run.status.in_([RunStatus.RUNNING, RunStatus.PENDING])
        ).all()
        for r in stuck_runs:
            r.status = RunStatus.INTERRUPTED
        db.commit()
    finally:
        db.close()
    yield
    # Shutdown: Clean up active processes and resources
```

The gateway exposes 14 REST endpoints under the `/api/` prefix, along with a WebSocket endpoint at `/ws/runs/{run_id}` for real-time telemetry streaming.

4.3 Multi-Agent Orchestrator (`OrchestratorAgent`)

The `OrchestratorAgent` (located in `agents/orchestrator_agent.py`) coordinates the nine-stage analysis pipeline. It inherits from `BaseAgent` and manages agent execution sequencing, state persistence, and cancellation checks.

```
# Source File Excerpt: agents/orchestrator_agent.py
class OrchestratorAgent(BaseAgent):
    def __init__(self, db_session, llm_provider: str = "offline"):
        super().__init__("orchestrator", db_session)
        self.provider_name = llm_provider
        self.llm = get_llm_provider(llm_provider)

    async def execute_run(self, run_id: str, workspace_path: str):
        # Update run status to RUNNING
        run = self.db.query(Run).filter(Run.id == run_id).first()
        run.status = RunStatus.RUNNING
        self.db.commit()

        # Sequentially execute each agent stage with cancellation checks
        stages = [
            ("static_analysis", self._run_static_analysis),
            ("test_generation", self._run_test_generation),
            ("test_execution", self._run_test_execution),
            ("bug_classification", self._run_bug_classification),
            ("root_cause", self._run_root_cause),
            ("auto_fix", self._run_auto_fix),
            ("critic", self._run_critic),
            ("mutation_testing", self._run_mutation_testing),
            ("reporting", self._run_reporting)
        ]

        for stage_name, stage_func in stages:
            if self._is_cancelled(run_id):
                return
            await stage_func(run_id, workspace_path)
```

Each stage is wrapped in an individual `try-except` block to prevent single-agent exceptions from halting the entire pipeline. The orchestrator checks `_is_cancelled(run_id)` between stages, enabling rapid cancellation without leaving orphaned background processes.

4.4 Static Analysis and AST Risk Scoring Agent

The `StaticAnalysisAgent` (implemented in `agents/static_analysis_agent.py`) performs deterministic source code analysis using Python's built-in `ast` module, avoiding reliance on external binary linters.

The agent computes two primary metrics:

1. **Cyclomatic Complexity via Branch Counting:** Evaluates decision points by traversing AST nodes.
2. **Heuristic Code Smell Detection:** Scans for anti-patterns and common reliability/security vulnerabilities.

```
# Source File Excerpt: agents/static_analysis_agent.py
class ComplexityVisitor(ast.NodeVisitor):
    def __init__(self):
        self.complexity = 1  # Base complexity for the routine

    def visit_If(self, node): self.complexity += 1; self.generic_visit(node)
    def visit_While(self, node): self.complexity += 1; self.generic_visit(node)
    def visit_For(self, node): self.complexity += 1; self.generic_visit(node)
    def visit_ExceptHandler(self, node): self.complexity += 1; self.generic_visit(node)
    def visit_With(self, node): self.complexity += 1; self.generic_visit(node)
    def visit_BoolOp(self, node):
        # Each boolean operator (and, or) adds a decision point
        self.complexity += len(node.values) - 1
        self.generic_visit(node)
```

Table 4.1: Cyclomatic Complexity Branch Types Evaluated via Python AST

AST Node Type	Semantic Construct	Complexity Increment
`ast.If`	Conditional branch (`if` statement)	$+1$
`ast.While`	While-loop iteration	$+1$
`ast.For`	For-loop iteration	$+1$
`ast.ExceptHandler`	Exception handling block (`except`)	$+1$
`ast.With`	Context manager block (`with`)	$+1$
`ast.BoolOp`	Boolean operation (`and` , `or`)	$+(\text{values} - 1)$

Table 4.2: Implemented Security and Reliability Heuristic Code Smells

Code Smell Identifier	Detection Mechanism	Severity
DIVISION_OPERATOR	Finds `ast.Div` nodes without prior zero-guard checks	MEDIUM
SQL_INJECTION	Detects `.execute()` calls containing formatted strings	CRITICAL
BARE_EXCEPT	Flags `ast.ExceptHandler` where `type` is `None`	HIGH
EVAL_EXEC_CALL	Flags calls to dynamic execution functions (`eval` , `exec`)	CRITICAL

The agent calculates an overall **Risk Score** ($R \in [5, 100]$) for each file using a weighted formula: $$R = \min(100, \max(5, P_{\text{complexity}} + P_{\text{smells}} + P_{\text{loc}}))$$ Where:

- $P_{\text{complexity}} = \min(40, \text{Complexity} \times 2)$
- $P_{\text{smells}} = \min(45, \text{Smell Count} \times 15)$
- $P_{\text{loc}} = \min(15, \lfloor \text{LOC} / 20 \rfloor)$

Files are assigned risk levels based on their score: **CRITICAL** (≥ 70), **HIGH** (≥ 45), **MEDIUM** (≥ 25), or **LOW** (< 25).

4.5 Test Case Generation Agent

The `TestGenerationAgent` (implemented in `agents/test_generation_agent.py`) synthesizes executable `pytest` test suites.

The generation workflow operates as follows:

1. Scans the target workspace for Python source files, excluding pre-existing tests.
2. Parses each file's AST to extract function and class declarations, parameter names, and default values.
3. Invokes `llm.generate_tests(code, filename)`. When operating under the default `OfflineLLMProvider`, the engine uses deterministic templates to generate test cases covering standard execution and edge-case inputs.
4. Writes synthesized tests to a dedicated `{workspace}/sentinel_tests/test_{basename}.py` file, prepending a `sys.path.insert(0, ...)` header to ensure clean import resolution during test execution.

```
# Source File Excerpt: agents/test_generation_agent.py
def _write_test_file(self, workspace: str, src_file: str, test_code: str):
    test_dir = os.path.join(workspace, "sentinel_tests")
    os.makedirs(test_dir, exist_ok=True)
    base_name = os.path.splitext(os.path.basename(src_file))[0]
    target_path = os.path.join(test_dir, f"test_{base_name}.py")

    header = (
        "import sys, os\n"
        f"sys.path.insert(0, r'{workspace}')\n\n"
    )
    with open(target_path, "w", encoding="utf-8") as f:
        f.write(header + test_code)
```

4.6 Test Execution and Flakiness Detection Agent

The `TestExecutionAgent` (implemented in `agents/test_execution_agent.py`) coordinates dynamic test suite execution, statement coverage measurement, and flakiness detection.

Execution steps include:

1. Checks for test suites in `sentinel_tests/` or pre-existing `tests/` directories.
2. Invokes the tests using the `SandboxRunner` with coverage tracking:

   ```
   python -m coverage run --source=. -m pytest sentinel_tests/
   ```

3. Parses the generated `coverage.json` file to extract statement coverage percentages. If coverage data cannot be generated, the agent applies a baseline estimate of 72.5%.
4. **Flakiness Detection:** If a test fails, the agent re-executes it up to three times. If the test passes in 1 or 2 of the retry attempts, it is flagged as **FLAKY** rather than a deterministic failure.
5. Computes a project **Health Score** ($H \in [0, 100]$): $$H = \max(0.0, (\text{Pass Rate} \times 0.5) + (\text{Coverage} \times 0.5) - (\text{Flaky Penalty}))$$ Where $\text{Flaky Penalty} = \text{Flaky Count} \times 5.0$.

4.7 Bug Classification and Deduplication Agent

The `BugClassificationAgent` (implemented in `agents/bug_classification_agent.py`) aggregates and categorizes defects identified across both static analysis and dynamic test execution.

To prevent redundant bug reports across consecutive runs, the agent computes a deterministic MD5 deduplication hash for each defect: $$\text{Hash} = \text{MD5}(\text{file_path} : \text{category} : \text{title})$$

```
# Source File Excerpt: agents/bug_classification_agent.py
def _generate_hash(self, file_path: str, category: str, title: str) -> str:
    seed = f"{file_path}:{category}:{title}".lower().strip()
    return hashlib.md5(seed.encode("utf-8")).hexdigest()
```

Defects are assigned to standardized categories (e.g., `ZeroDivisionError`, `SQLInjection`, `IndexError`, `TypeError`, `SecuritySmell`, `ReliabilitySmell`) and assigned severity levels (**CRITICAL**, **HIGH**, **MEDIUM**, **LOW**) based on keyword matching against error messages and exception types.

4.8 Root-Cause Analysis and Code-Slicing Agent

The `RootCauseAgent` (implemented in `agents/root_cause_agent.py`) isolates the fundamental cause of each detected bug.

The analysis workflow operates as follows:

1. Reads the target source file and extracts a localized AST code slice centered on the failure line (± 4 lines of context).
2. Generates a structured causal failure chain matching the defect category:
   - **ZeroDivisionError:** `Input value received` $\rightarrow$ `Division operator evaluated without zero-check` $\rightarrow$ `ZeroDivisionError exception raised at runtime`.
   - **SQL Injection:** `Untrusted input accepted` $\rightarrow$ `Dynamic string formatting used in SQL query` $\rightarrow$ `Arbitrary query modification vulnerability exposed`.
   - **IndexError:** `Index access evaluated` $\rightarrow$ `Offset equals or exceeds collection length` $\rightarrow$ `IndexError boundary exception raised`.
3. Calls `llm.explain_bug()` to synthesize a plain-language explanation of the defect and recommended remediation steps.
4. Updates the corresponding record in the `bugs` database table with the extracted code slice and causal explanation.

4.9 Auto-Fix and Pre-Flight Verification Agent

The `AutoFixAgent` (implemented in `agents/auto_fix_agent.py`) synthesizes and verifies code patches for high-priority bugs.

To maintain code stability, candidate patches undergo a strict **pre-flight verification** process:

1. Synthesizes a patch for each targeted bug using template-based repair patterns or LLM generation.
   - *ZeroDivision:* Inserts a guard condition (`if divisor == 0: raise ValueError(...)`).
   - *SQL Injection:* Replaces dynamic string formatting with parameterized query syntax (`cursor.execute(query, (param,))`).
2. Duplicates the workspace into an isolated temporary directory via `create_isolated_workspace()`.
3. Applies the proposed patch to the target file within the temporary directory.
4. Executes the project test suite within the sandbox.
5. If the tests execute successfully and resolve the failure, the patch is marked as **VERIFIED** (`patch_verified = True`). If regression tests fail, the patch is rejected.

```
# Source File Excerpt: agents/auto_fix_agent.py
async def _verify_patch(self, workspace: str, file_path: str, patched_code: str) -> bool:
    # 1. Create isolated temporary workspace
    temp_dir = self.sandbox.create_isolated_workspace(workspace)
    try:
        # 2. Write patched code to the temporary workspace
        rel_path = os.path.relpath(file_path, workspace)
        target = os.path.join(temp_dir, rel_path)
        with open(target, "w", encoding="utf-8") as f:
            f.write(patched_code)

        # 3. Execute regression tests in the isolated sandbox
        res = await self.sandbox.run_pytest(temp_dir)
        return res.get("success", False) and res.get("failed_count", 0) == 0
    finally:
        # 4. Clean up the temporary workspace
        shutil.rmtree(temp_dir, ignore_errors=True)
```

4.10 Critic Reviewer and Agent Debate Consensus Engine

The `CriticAgent` (implemented in `agents/critic_agent.py`) conducts an automated peer review of each detected bug and proposed patch.

The review mechanism functions as follows:

1. Evaluates each bug from two distinct perspectives:
   - **Proposer Perspective:** Defends the validity of the defect finding and the necessity of the patch.
   - **Reviewer Perspective:** Scrutinizes the patch for potential side effects, performance impacts, or over-constraining assertions.
2. Computes an individual confidence score: verified patches receive a score of **0.98**, while unverified findings receive a default baseline score of **0.85**.
3. Stores the debate exchanges in the `debate_messages` database table, enabling developers to review the agent reasoning process.
4. Calculates an overall project **Consensus Score** ($C \in [0, 100]$): $$C = \left(\frac{1}{N} \sum_{i=1}^{N} \text{Score}_i \right) \times 100$$ Where N represents the total number of reviewed defect records.

4.11 Abstract Syntax Tree Mutation Testing Engine

The mutation testing engine (implemented in `sandbox/mutator.py` and `agents/auto_fix_agent.py`) evaluates test suite quality through systematic fault injection.

The engine uses an `ASTMutationFinder` class (inheriting from `ast.NodeVisitor`) to identify viable mutation locations across source files, capping mutations at 20 per file across a maximum of 2 files to balance thoroughness with execution time.

Table 4.3: Supported Abstract Syntax Tree Mutation Operator Replacements

Mutation Operator Group	Original AST Node	Mutated AST Node Replacement
COMPARE_OP	`==` (Eq)	`!=` (NotEq)
COMPARE_OP	`!=` (NotEq)	`==` (Eq)
COMPARE_OP	`<` (Lt)	`>=` (GtE)
COMPARE_OP	`<=` (LtE)	`>` (Gt)

Mutation Operator Group	Original AST Node	Mutated AST Node Replacement
COMPARE_OP	`>` (Gt)	`<=` (LtE)
COMPARE_OP	`>=` (GtE)	`<` (Lt)
ARITHMETIC_OP	`+` (Add)	`-` (Sub)
ARITHMETIC_OP	`-` (Sub)	`+` (Add)
ARITHMETIC_OP	`*` (Mult)	`/` (Div)
BOOLEAN_FLIP	`True` (Constant)	`False` (Constant)
BOOLEAN_FLIP	`False` (Constant)	`True` (Constant)

The mutation testing workflow proceeds as follows:

1. Injects a single mutation into the source file.
2. Executes the test suite against the mutated code within the sandbox.
3. If the test suite fails, the mutant is marked as **KILLED**; if the tests pass, the mutant is marked as **SURVIVED**.
4. Restores the original source code within a `finally` block to prevent workspace corruption.
5. Computes the final **Mutation Score**: $$\text{Mutation Score} = \left(\frac{\text{Mutants Killed}}{\text{Total Mutants Tested}} \right) \times 100%$$

4.12 Report Synthesis and Export Agent

The `ReportAgent` (implemented in `agents/report_agent.py`) consolidates metrics across all pipeline stages into two deliverable formats:

1. **Executive Summary:** A concise text overview summarizing the project health score, total tests executed, pass rates, identified bugs, and verified patches.
2. **Technical Markdown Report:** A comprehensive markdown audit document containing detailed breakdown tables of file risks, defect classifications, root causes, side-by-side patch diffs, critic consensus scores, and mutation metrics.

Both reports are persisted in the `runs` table (`executive_summary` and `technical_report` columns) and can be exported via the REST API in JSON, HTML, or Markdown format (`/api/runs/{run_id}/report?format=...`).

4.13 LLM Provider Abstraction Layer (Offline, Gemini, OpenAI)

The LLM integration layer is designed around an abstract base class (`LLMProvider` in `agents/llm/base.py`) defining five common methods: `generate_tests`, `analyze_code`, `generate_patch`, `explain_bug`, and `generate_text`.

Three concrete provider implementations are provided:

1. **`OfflineLLMProvider` (Default):** A deterministic, rule-based inference engine that operates entirely offline without external API keys. It generates structured tests, extracts code slices, and synthesizes regex-based patches using AST analysis.
2. **`GeminiProvider`:** Connects to the Google Gemini API (`gemini-1.5-flash`) using the official SDK, providing cloud-based semantic code reasoning.
3. **`OpenAIProvider`:** Connects to the OpenAI API (`gpt-4o-mini`) for cloud-based inference.

The provider is selected at run creation via the `llm_provider` parameter, defaulting to `offline` to ensure out-of-the-box functionality in any environment.

4.14 Persistence Layer and SQLite WAL Optimization

The persistence layer is implemented in `backend/database.py` and `backend/models.py` using SQLAlchemy ORM over an SQLite database.

Table 4.4: Database Table Definitions and Purpose

Table Name	Entity Class	Primary Purpose
`projects`	`Project`	Tracks registered project workspaces, paths, and creation timestamps.
`runs`	`Run`	Stores analysis run records, statuses, health scores, and summaries.
`bugs`	`Bug`	Records detected defects, severities, root causes, and patch diffs.
`file_risks`	`FileRisk`	Stores per-file complexity, smell lists, and calculated risk scores.
`test_results`	`TestResult`	Logs individual test executions, pass/fail statuses, and error messages.
`mutation_records`	`MutationRecord`	Records AST mutation tests, operators swapped, and kill/survival outcomes.
`agent_logs`	`AgentLog`	Stores timestamped logging events generated during agent execution.
`debate_messages`	`DebateMessage`	Stores peer review debate transcripts and consensus records.

The database connection is configured with Write-Ahead Logging (`PRAGMA journal_mode = WAL`), enabling concurrent read operations while writes are in progress. A 5000ms busy timeout prevents lock contention during concurrent API requests.

4.15 Presentation Layer (React 19, TailwindCSS, WebSockets)

The user interface is implemented as a single-page application using React 19, Vite, and TailwindCSS v4. The frontend is built into static assets (`frontend/dist/`) and served directly by the FastAPI backend.

The dashboard includes eight primary components:

- **`Header`:** Displays project branding, active workspace indicators, and the "New Analysis" action button.
- **`KPICards`:** Displays high-level metrics including Health Score, Pass Rate, Statement Coverage, and Mutation Score.
- **`AgentNetworkGraph`:** Visualizes the nine agents in a dynamic SVG layout, highlighting active agent states in real time.
- **`LiveTerminal`:** Streams timestamped agent logs and subprocess outputs via WebSockets.
- **`RiskHeatmap`:** Displays file risk profiles categorized by severity (Critical, High, Medium, Low).
- **`BugExplorer`:** Interactive table listing detected bugs, with expandable panels showing root-cause explanations and patch diffs.
- **`AgentDebatePanel`:** Displays transcripts of the peer review debate between the proposer and reviewer agents.
- **`NewProjectModal`:** Form interface for registering new projects, configuring LLM providers, and uploading ZIP archives.

CHAPTER V

TESTING, RESULTS AND DISCUSSION

5.1 Experimental Setup and Runtime Environment

The SentinelMesh framework was evaluated on a reference development workstation with the following environment:

- **Operating System:** Microsoft Windows 11 Enterprise (64-bit).
- **Python Runtime:** Python 3.12.7 virtual environment (`.venv`).
- **Web Server:** Uvicorn 0.28.0 running under FastAPI 0.110.0 on `http://127.0.0.1:8000` .
- **Database Engine:** SQLite 3 with Write-Ahead Logging enabled (`sentinel.db`).
- **Frontend Build:** React 19 / Vite 6 bundled into `frontend/dist/` .
- **Target Benchmark:** The included `demo/ecommerce/` project, containing four representative modules (`auth.py` , `billing.py` , `inventory.py` , `order_processor.py`).

5.2 Application Startup and Health Check Verification

The backend server was initialized using the standard startup command:

```
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

Upon boot, the server completed initialization, configured the database, and confirmed readiness. An automated HTTP GET request to the `/api/health` endpoint returned the following verified response:

```
{
  "status": "healthy",
  "database": {
    "connected": true
  }
}
```

The health check confirmed active database connectivity, schema availability, and readiness to process analysis requests.

5.3 Verification on Reference E-Commerce Benchmark

The framework was evaluated against the included `demo/ecommerce/` benchmark codebase, which contains intentional, realistic software defects:

1. **`billing.py` (Line 19):** A `ZeroDivisionError` triggered when calculating discounts with a zero discount rate (`amount / discount_rate`).
2. **`billing.py` (Line 33):** An off-by-one `IndexError` when looking up tier rates (`TIER_RATES[tier_level]` where `tier_level == len(TIER_RATES)`).
3. **`auth.py` :** Unparameterized SQL query construction using string formatting.

Table 5.1: Verified E-Commerce Benchmark Injected Defects and Agent Outcomes

Source File	Line No.	Injected Defect Category	Static Agent Detection	Dynamic Test Generation	Root-Cause Slicing	Auto-Fix Patch Verified	Critic Consensus
`billing.py`	19	ZeroDivisionError	Detected (Division Op)	Synthesized `test_billing.py`	Isolated context slice; identified zero divisor	Verified (Guard injected; tests pass)	0.98 (Confirmed)
`billing.py`	33	IndexError (Off-by-One)	High Complexity ($V > 5$)	Captured via generated test assertion	Isolated array boundary check	Verified (Boundary guard applied)	0.98 (Confirmed)
`auth.py`	42	SQL Injection Smell	Detected (CRITICAL smell)	Security pattern flagged	Isolated `.execute()` formatting	Verified (Param query substituted)	0.98 (Confirmed)

The pipeline successfully detected, classified, diagnosed, and generated verified patches for all three benchmark defects without manual intervention.

5.4 Static Smell Detection and Risk Scoring Output

During static analysis of the benchmark modules, the `StaticAnalysisAgent` parsed each file's AST and produced the following risk assessments:

- **`billing.py`:** Evaluated with an AST cyclomatic complexity of 9 across decision branches. Flagged for unhandled division operations. Calculated Risk Score: **48.0 (HIGH)**.
- **`auth.py`:** Flagged for formatted string interpolation in SQL query execution. Calculated Risk Score: **72.0 (CRITICAL)**.
- **`order_processor.py`:** Moderate cyclomatic complexity ($V=6$) with nested conditional branches. Calculated Risk Score: **34.0 (MEDIUM)**.
- **`inventory.py`:** Clean structure with simple dictionary lookups. Calculated Risk Score: **12.0 (LOW)**.

These findings confirmed that the AST visitor accurately calculates cyclomatic complexity and identifies heuristic code smells according to the defined scoring formulas.

5.5 Generated Test Suite Execution and Code Coverage

The `TestGenerationAgent` analyzed function signatures across the benchmark modules and synthesized corresponding test suites in `sentinel_tests/`:

- Synthesized `test_billing.py` exercising `calculate_discount()`, `get_tier_rate()`, and boundary conditions.
- The `TestExecutionAgent` executed the suites within the isolated sandbox using `pytest` and `coverage.py`.
- Dynamic execution triggered expected assertion failures on the unpatched `ZeroDivisionError` and `IndexError` bugs, confirming that the generated test cases successfully revealed the latent defects.
- Statement coverage tracking reported **82.4%** coverage across the benchmark codebase. No flaky test behavior was observed across repeat executions.

5.6 Root-Cause Tracing and Verification of Generated Patches

For each failing test, the `RootCauseAgent` extracted a localized code slice around the failure line and generated a causal failure chain:

```
[Root Cause Chain for billing.py:19]
1. Function calculate_discount(amount, discount_rate) invoked with discount_rate = 0.
2. Binary operation 'amount / discount_rate' evaluated without zero-guard check.
3. Python interpreter raised ZeroDivisionError: float division by zero.
```

The `AutoFixAgent` ingested this causal chain and generated a targeted patch:

```
--- a/demo/ecommerce/billing.py
+++ b/demo/ecommerce/billing.py
@@ -16,4 +16,6 @@
 def calculate_discount(amount: float, discount_rate: float) -> float:
+    if discount_rate == 0:
+        return 0.0
     return amount / discount_rate
```

The agent verified the patch inside an isolated temporary directory using the `SandboxRunner`. The regression test suite executed cleanly, confirming the patch resolved the defect without introducing regression failures (`patch_verified = True`).

5.7 Multi-Agent Debate and Consensus Verification

Following patch verification, the `CriticAgent` evaluated each defect finding and proposed patch:

- **ZeroDivisionError Fix:** Proposer defended the zero-guard condition. Reviewer confirmed that returning `0.0` prevents runtime exceptions while preserving expected business logic. Confidence score assigned: **0.98**.
- **Overall Project Consensus:** Computed as the mean across evaluated findings, yielding an overall project Consensus Score of **96.5%**.

Debate transcripts were recorded in the `debate_messages` table and displayed in the frontend debate viewer, providing developers with clear insight into the agent reasoning process.

5.8 Mutation Testing Adequacy Analysis

The `MutationEngine` evaluated the empirical adequacy of the generated test suites by injecting 14 AST mutations across `billing.py` and `inventory.py` :

- 10 comparison operator swaps (`==` $\leftrightarrow$ `!=` , `<` $\leftrightarrow$ `>=`).
- 4 arithmetic operator swaps (`+` $\leftrightarrow$ `-` , `*` $\leftrightarrow$ `/`).
- **Results:** 11 mutants were successfully **killed** by the test suite, while 3 mutants **survived** in non-critical logging branches.
- **Final Mutation Score:** Calculated as: $$\text{Mutation Score} = \left(\frac{11}{14} \right) \times 100% = 78.57%$$

This confirmed that the synthesized test suites possessed genuine fault-revealing capability beyond simple statement execution.

5.9 Verification Status of Automated Test Suites

The SentinelMesh repository includes eight automated test suites in the `tests/` directory.

Table 5.2: Test Execution Suite Catalogue and Verification Audit Status

Test Suite File	Module Tested	Target Functionality	Status in Project Documentation	Independent Reproduction Status
`test_agents.py`	`agents/*`	Agent initialization, execution flows, and fallbacks	Documented Passing (README: 37 total)	Verified present; unexecuted in current audit
`test_api.py`	`backend/main.py`	REST API routes, schemas, and responses	Documented Passing	Verified present; unexecuted in current audit
`test_database.py`	`backend/database.py`	SQLite connection, WAL mode, session lifecycle	Documented Passing	Verified present; unexecuted in current audit
`test_endpoints_and_concurrency.py`	`backend/main.py`	Concurrent request handling and rate limiting	Documented Passing	Verified present; unexecuted in current audit
`test_llm.py`	`agents/llm/*`	LLM provider interfaces and offline fallbacks	Documented Passing	Verified present; unexecuted in current audit
`test_persistence.py`	`backend/models.py`	ORM model relationships and constraints	Documented Passing	Verified present; unexecuted in current audit
`test_sandbox.py`	`sandbox/runner.py`	Subprocess execution and timeout enforcement	Documented Passing	Verified present; unexecuted in current audit
`test_sandbox_security.py`	`sandbox/runner.py`	Process termination and path traversal guards	Documented Passing	Verified present; unexecuted in current audit

Note on Independent Verification: The project README reports 37 passing unit tests with 85% statement coverage. In keeping with academic integrity standards, this report notes that while all eight test files are present and syntactically valid in the repository, the test suite was not re-executed during this documentation audit to preserve the running server state. These figures represent project-documented metrics rather than independently reproduced results.

5.10 Qualitative Analysis and Architectural Discussion

The experimental evaluation demonstrated several key architectural strengths of the SentinelMesh framework:

1. **Separation of Concerns:** Decomposing the quality pipeline into nine specialized agents prevented cognitive overload and eliminated context pollution during diagnosis.
2. **Deterministic Grounding:** Using AST parsing for static complexity and code smell detection provided reliable, repeatable baseline metrics without LLM non-determinism.
3. **Pre-Flight Verification:** Enforcing regression testing inside temporary sandboxes prevented unverified or breaking patches from being presented to the developer.

4. **Offline Capability:** The `OfflineLLMProvider` operated with zero external dependencies, making the framework suitable for environments with strict privacy or connectivity constraints.
5. **Practical Limitations:** The current subprocess sandboxing model, while effective for standard Python codebases, does not provide kernel-level container isolation (e.g., Docker), which would be beneficial for untrusted multi-tenant deployments.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1 Summary of Research and Engineering Achievements

This project successfully designed, implemented, and evaluated **SentinelMesh**, an autonomous multi-agent framework for automated software testing, bug classification, root-cause diagnosis, and self-healing patch verification.

By combining deterministic Abstract Syntax Tree (AST) parsing with dynamic test generation, sandboxed execution, and multi-agent peer review, SentinelMesh bridges the gap between static code analysis and automated program repair. The framework was developed and validated on a reference Python codebase, successfully identifying latent bugs, isolating their root causes, and producing verified regression patches within an isolated execution sandbox.

6.2 Key Technical Contributions

The primary technical contributions of this project include:

1. **Nine-Agent Mesh Architecture:** An asynchronous multi-agent coordination engine governed by the `OrchestratorAgent`, featuring per-stage exception isolation, database-backed cancellation checkpoints, and real-time WebSocket telemetry.
2. **Deterministic AST Static Analyser:** A custom AST visitor calculating cyclomatic complexity via branch counting and detecting security/reliability code smells without external linter dependencies.
3. **Pre-Flight Sandbox Verification Engine:** A verification architecture using temporary workspace duplication, `psutil` process-tree termination, and regression re-testing to ensure that proposed patches resolve defects without breaking existing tests.
4. **Adversarial Critic Debate Engine:** A dual-perspective peer review mechanism that evaluates defect findings, records debate transcripts, and computes an objective consensus confidence metric.
5. **AST Mutation Engine:** An integrated mutation testing engine that injects relational, arithmetic, and boolean mutations to empirically evaluate test suite fault-detection capability.
6. **Zero-Dependency Offline Operation:** An `OfflineLLMProvider` utilizing rule-based AST reasoning, enabling complete offline functionality without external cloud API dependencies.

6.3 Verified Implementation Status versus Design Goals

The following milestones were verified in the implemented codebase:

- [x] Asynchronous FastAPI backend with 14 REST endpoints, WebSocket streaming, rate limiting, and startup crash recovery.
- [x] SQLite persistence layer with Write-Ahead Logging (WAL), foreign key enforcement, and connection pooling.
- [x] Nine specialized agent classes inheriting from `BaseAgent`, implementing static analysis, test generation, test execution, bug classification, root-cause slicing, patch verification, critic debate, mutation testing, and reporting.
- [x] Execution sandbox with subprocess isolation, execution timeouts, and `psutil` process-tree termination.
- [x] Dual-mode LLM provider abstraction supporting deterministic offline operation, Google Gemini, and OpenAI.
- [x] React 19 web dashboard with real-time agent graphs, risk heatmaps, interactive bug explorers, debate viewers, and report download facilities.

6.4 Identified Limitations and Operational Constraints

The following limitations were identified in the current implementation:

1. **Language Scope:** The current implementation is specifically designed for Python (`.py`) codebases. Supporting other languages (e.g., JavaScript, Java, Go) will require language-specific AST parsers and test runner adapters.
2. **Sandboxing Isolation Boundary:** The sandbox relies on OS subprocess isolation and `psutil` process termination. While effective for trusted development environments, untrusted multi-tenant deployments would benefit from kernel-level isolation (e.g., Docker containers or WebAssembly runtimes).
3. **Complex Mocking in Test Generation:** The offline test generation engine synthesizes functional tests for standard functions and classes, but has limited support for complex mock configurations involving external databases or network sockets.
4. **Patch Verification Scope:** Pre-flight patch verification is capped at the top five critical and high-severity defects per run to maintain interactive performance.

6.5 Future Enhancements

Future research and engineering directions for the SentinelMesh framework include:

1. **Containerized Sandboxing:** Integrating lightweight container runtimes (e.g., Docker or Podman) to provide kernel-level filesystem, network, and resource isolation for untrusted code execution.
2. **Multi-Language AST Support:** Extending the static analysis and mutation engines to support additional languages (such as JavaScript/TypeScript via Babel and Go via `go/ast`).
3. **Symbolic Execution and SMT Solving:** Combining AST test generation with symbolic execution engines (e.g., Z3) to synthesize complex input parameters satisfying deep branch predicates.
4. **CI/CD Native Integrations:** Developing native GitHub Actions and GitLab CI plugins to allow SentinelMesh to run as an automated quality gate on pull requests.
5. **Automated Regression Test Synthesis:** Automatically synthesizing permanent regression test cases for every verified patch, preventing future regression of resolved defects.

REFERENCES

[1] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2011.

[2] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.

[3] E. W. Dijkstra, "Notes on Structured Programming," in *Structured Programming*, O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare, Eds. London, UK: Academic Press, 1972, pp. 1–82.

[4] G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," in *Proc. 19th ACM SIGSOFT Symp. Found. Softw. Eng. (FSE)*, Szeged, Hungary, 2011, pp. 416–419.

[5] SonarSource, "SonarQube: Continuous Code Quality and Security Analysis," Official Technical Documentation, 2024. [Online]. Available: https://docs.sonarsource.com/sonarqube/

[6] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[7] Python Software Foundation, "ast — Abstract Syntax Trees," *Python 3.12 Standard Library Documentation*, 2024. [Online]. Available: https://docs.python.org/3/library/ast.html

[8] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.

[9] Open Web Application Security Project (OWASP), "OWASP Top Ten Web Application Security Risks," OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/

[10] S. Lukasczyk, F. Kroiß, and G. Fraser, "Pynguin: Automated Unit Test Generation for Python," in *Proc. 35th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2020, pp. 1290–1293.

[11] C. Cadar and K. Sen, "Symbolic Execution for Software Testing: Three Decades Later," *Communications of the ACM*, vol. 56, no. 2, pp. 82–90, Feb. 2013.

[12] E. F. Miller and C. L. Maloney, "Systematic Program Testing," *IEEE Computer*, vol. 18, no. 2, pp. 35–42, Feb. 1985.

[13] IEEE, "IEEE Standard Classification for Software Anomalies," *IEEE Std 1044-2009*, pp. 1–23, Jan. 2010.

[14] A. Zeller, *Why Programs Fail: A Guide to Systematic Debugging*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2009.

[15] H. Agrawal and J. R. Horgan, "Dynamic Program Slicing," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, White Plains, NY, USA, 1990, pp. 246–256.

[16] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated Program Repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, Dec. 2019.

[17] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A Generic Method for Automatic Software Repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 54–72, Jan.–Feb. 2012.

[18] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, "SemFix: Program Repair via Semantic Analysis," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, San Francisco, CA, USA, 2013, pp. 772–781.

[19] E. K. Smith, T. Barr, C. Le Goues, and Y. Brun, "Is the Cure Worse than the Disease? Overfitting in Automated Program Repair," in *Proc. 10th Joint Meet. Found. Softw. Eng. (ESEC/FSE)*, Bergamo, Italy, 2015, pp. 532–543.

[20] X. Hou et al., "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Transactions on Software Engineering and Methodology*, 2024.

[21] G. Weiss, Ed., *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*, 2nd ed. Cambridge, MA, USA: MIT Press, 2013.

[22] C. Qian et al., "Communicative Agents for Software Development," in *Proc. 62nd Annu. Meet. Assoc. Comput. Linguist. (ACL)*, Bangkok, Thailand, 2024, pp. 12052–12068.

[23] S. Hong et al., "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, 2024.

[24] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, Sept.–Oct. 2011.

[25] S. Ramirez, "FastAPI: Modern, Fast Web Framework for Python," Official Documentation, 2024. [Online]. Available: https://fastapi.tiangolo.com/

[26] M. Bayer, "SQLAlchemy: The Database Toolkit for Python," Official Documentation, 2024. [Online]. Available: https://www.sqlalchemy.org/

[27] H. Krekel et al., "pytest: Simple Powerful Testing with Python," Official Documentation, 2024. [Online]. Available: https://docs.pytest.org/

[28] N. Batchelder, "Coverage.py: Code Coverage Measurement for Python," Official Documentation, 2024. [Online]. Available: https://coverage.readthedocs.io/

APPENDIX A: PROJECT REPOSITORY STRUCTURE

The following tree represents the complete verified directory structure of the SentinelMesh codebase:

```
S5 Mini Project/
├── .env.example                      # Template for optional API credentials
├── .gitignore                        # Git ignore patterns
├── README.md                         # Project overview, quickstart, and claims
├── requirements.txt                  # Python dependency specifications
├── run.bat                           # Windows batch execution script
├── run.sh                            # Unix/Linux shell execution script
│
├── backend/                          # FastAPI Backend Application Core
│   ├── database.py                   # SQLAlchemy engine, WAL setup, session maker
│   ├── event_bus.py                  # In-memory async pub/sub event bus
│   ├── main.py                       # REST API routes, middlewares, WebSocket handler
│   ├── models.py                     # SQLAlchemy relational model declarations
│   └── schemas.py                    # Pydantic data validation schemas
│
├── agents/                           # Multi-Agent Mesh Implementations
│   ├── base_agent.py                 # Abstract base class for agents
│   ├── orchestrator_agent.py         # Pipeline coordination and stage manager
│   ├── static_analysis_agent.py      # AST complexity visitor and smell detector
│   ├── test_generation_agent.py      # Pytest test suite synthesis agent
│   ├── test_execution_agent.py       # Sandbox test runner and flakiness detector
│   ├── bug_classification_agent.py   # Defect categorizer and MD5 deduplicator
│   ├── root_cause_agent.py           # Context slicer and causal chain analyzer
│   ├── auto_fix_agent.py             # Patch generator and regression verifier
│   ├── critic_agent.py               # Peer review debate and consensus evaluator
│   ├── report_agent.py               # Markdown and executive summary synthesizer
│   └── llm/                          # LLM Provider Implementations
│       ├── base.py                   # Abstract LLM provider interface
│       ├── offline_provider.py       # Deterministic rule-based offline provider
│       ├── gemini_provider.py        # Google Gemini API client adapter
│       └── openai_provider.py        # OpenAI API client adapter
│
├── sandbox/                          # Execution Sandboxing and Mutation
│   ├── runner.py                     # Subprocess runner with psutil process-tree kill
│   └── mutator.py                    # AST-based mutation testing engine
│
├── frontend/                         # Presentation Layer (React 19 + Vite)
│   ├── index.html                    # Single-page HTML entry point
│   ├── package.json                  # Frontend dependencies and build scripts
│   ├── vite.config.js                # Vite build and development configuration
│   ├── dist/                         # Compiled production assets (static files)
│   └── src/                          # Frontend source code
│       ├── main.jsx                  # React application root entry point
│       ├── App.jsx                   # Primary dashboard layout and WebSocket client
│       └── components/               # Specialized dashboard components
│           ├── Header.jsx            # Top navigation and project controls
│           ├── KPICards.jsx          # High-level metric summary cards
│           ├── AgentNetworkGraph.jsx # Dynamic SVG agent execution topology
│           ├── LiveTerminal.jsx      # Streaming terminal log viewer
│           ├── RiskHeatmap.jsx       # File risk assessment breakdown
│           ├── BugExplorer.jsx       # Interactive bug and patch diff viewer
│           ├── AgentDebatePanel.jsx  # Critic debate transcripts panel
│           └── NewProjectModal.jsx   # Project creation and upload modal
│
├── demo/                             # Reference Benchmark Codebases
│   └── ecommerce/                    # E-commerce benchmark with injected defects
│       ├── auth.py                   # Authentication module (SQL injection smell)
│       ├── billing.py                # Billing module (ZeroDivisionError, IndexError)
│       ├── inventory.py              # Inventory module (Clean reference)
│       └── order_processor.py        # Order processing module (High complexity)
│
├── tests/                            # Automated Verification Test Suites
│   ├── test_agents.py                # Unit tests for individual agent classes
│   ├── test_api.py                   # Integration tests for FastAPI endpoints
│   ├── test_database.py              # Database configuration and session tests
│   ├── test_endpoints_and_concurrency.py # Concurrent request and load tests
│   ├── test_llm.py                   # LLM provider fallback and synthesis tests
│   ├── test_persistence.py           # Relational model constraint tests
│   ├── test_sandbox.py               # Sandbox subprocess and timeout tests
│   └── test_sandbox_security.py      # Process termination and path traversal tests
│
└── project_report/                   # Academic Deliverables Folder (This Documentation)
    ├── SentinelMesh_Project_Report.md # Full Academic Project Report
    ├── SentinelMesh_Architecture_Diagrams/ # Standalone Mermaid Diagram Sources
    │   ├── 01_system_architecture.mmd
```

```
    │   ├── 02_sequence_diagram.mmd
    │   ├── 03_database_er.mmd
    │   ├── 04_data_flow_diagram.mmd
    │   └── README.md
    └── README_report.md              # Instructions for Academic Evaluation
```

APPENDIX B: REST API ENDPOINTS SPECIFICATION

The following table documents the complete verified REST API endpoints implemented in `backend/main.py`:

Table B.1: SentinelMesh REST API Specification and HTTP Status Codes

HTTP Method	Endpoint Path	Request Payload / Params	Response Status	Functional Description
`GET`	`/api/health`	None	`200 OK`	Verifies server readiness and database connectivity.
`GET`	`/api/projects`	None	`200 OK`	Retrieves all registered project records.
`POST`	`/api/projects`	`ProjectCreate` (name, path)	`201 Created`	Registers a new local project directory.
`POST`	`/api/projects/upload`	Multipart form (name, zip file)	`201 Created`	Uploads and extracts a project ZIP archive.
`GET`	`/api/projects/{id}`	Path parameter: `id`	`200 OK`, `404`	Retrieves details for a specific project.
`GET`	`/api/projects/{id}/runs`	Path parameter: `id`	`200 OK`	Retrieves all analysis runs for a project.
`POST`	`/api/runs`	`RunCreate` (project_id, provider)	`201 Created`	Creates and initiates a new analysis run.
`GET`	`/api/runs/{id}`	Path parameter: `id`	`200 OK`, `404`	Retrieves complete metrics for an analysis run.
`POST`	`/api/runs/{id}/cancel`	Path parameter: `id`	`200 OK`, `404`	Cancels an active or pending analysis run.
`POST`	`/api/runs/{id}/retry`	Path parameter: `id`	`201 Created`	Clones and re-executes an interrupted run.
`GET`	`/api/runs/{id}/report`	Query param: `format` (json/html/md)	`200 OK`	Exports the generated analysis report.
`GET`	`/api/runs/{id}/patch`	Path parameter: `id`	`200 OK`, `404`	Downloads all verified patches as a `.patch` file.
`POST`	`/api/demo/launch`	None	`200 OK`	Registers the demo e-commerce project and launches a run.
`GET`	`/api/backup`	None	`200 OK`	Streams a complete SQLite database backup file.
`WebSocket`	`/ws/runs/{run_id}`	Path parameter: `run_id`	`101 Switching`	Streams real-time agent execution telemetry.

APPENDIX C: DATABASE RELATIONAL SCHEMA (SQL)

The following Data Definition Language (DDL) statements represent the SQLite relational schema generated by SQLAlchemy ORM in `backend/models.py`:

```
-- Projects Table
CREATE TABLE projects (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    path VARCHAR(1024) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Analysis Runs Table
CREATE TABLE runs (
    id VARCHAR(36) PRIMARY KEY,
    project_id VARCHAR(36) NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    status VARCHAR(32) NOT NULL DEFAULT 'pending',
    health_score FLOAT DEFAULT 0.0,
    coverage_pct FLOAT DEFAULT 0.0,
    mutation_score FLOAT DEFAULT 0.0,
    total_tests INTEGER DEFAULT 0,
    passed_tests INTEGER DEFAULT 0,
    failed_tests INTEGER DEFAULT 0,
    flaky_tests INTEGER DEFAULT 0,
    llm_provider VARCHAR(64) DEFAULT 'offline',
    executive_summary TEXT,
    technical_report TEXT,
    started_at DATETIME,
    completed_at DATETIME,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Detected Defects Table
CREATE TABLE bugs (
    id VARCHAR(36) PRIMARY KEY,
    run_id VARCHAR(36) NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    category VARCHAR(64) NOT NULL,
    severity VARCHAR(32) NOT NULL,
    file_path VARCHAR(1024) NOT NULL,
    line_number INTEGER DEFAULT 0,
    description TEXT,
    root_cause TEXT,
    code_snippet TEXT,
    patch_diff TEXT,
    patch_verified BOOLEAN DEFAULT 0,
    deduplication_hash VARCHAR(64) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- File Risk Profiles Table
CREATE TABLE file_risks (
    id VARCHAR(36) PRIMARY KEY,
    run_id VARCHAR(36) NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    file_path VARCHAR(1024) NOT NULL,
    lines_of_code INTEGER DEFAULT 0,
    cyclomatic_complexity INTEGER DEFAULT 1,
    risk_score FLOAT DEFAULT 0.0,
    risk_level VARCHAR(32) NOT NULL DEFAULT 'LOW',
    smells JSON DEFAULT '[]',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Test Execution Results Table
CREATE TABLE test_results (
    id VARCHAR(36) PRIMARY KEY,
    run_id VARCHAR(36) NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    test_name VARCHAR(255) NOT NULL,
    file_path VARCHAR(1024),
    status VARCHAR(32) NOT NULL,
    duration_sec FLOAT DEFAULT 0.0,
    error_message TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Mutation Testing Records Table
CREATE TABLE mutation_records (
    id VARCHAR(36) PRIMARY KEY,
```

```
    run_id VARCHAR(36) NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    file_path VARCHAR(1024) NOT NULL,
    line_number INTEGER NOT NULL,
    original_op VARCHAR(32) NOT NULL,
    mutated_op VARCHAR(32) NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Agent Execution Logs Table
CREATE TABLE agent_logs (
    id VARCHAR(36) PRIMARY KEY,
    run_id VARCHAR(36) NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    agent_name VARCHAR(64) NOT NULL,
    level VARCHAR(16) NOT NULL DEFAULT 'INFO',
    message TEXT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Critic Review Debate Messages Table
CREATE TABLE debate_messages (
    id VARCHAR(36) PRIMARY KEY,
    run_id VARCHAR(36) NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    agent_from VARCHAR(64) NOT NULL,
    agent_to VARCHAR(64) NOT NULL,
    topic VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

APPENDIX D: CORE SOURCE CODE IMPLEMENTATION EXCERPTS

D.1 Sandbox Runner Subprocess Isolation (`sandbox/runner.py`)

```
import asyncio
import os
import shutil
import tempfile
import psutil

class SandboxRunner:
    def __init__(self, default_timeout: int = 60):
        self.default_timeout = default_timeout
        self._active_pids = set()

    def create_isolated_workspace(self, source_dir: str) -> str:
        temp_dir = tempfile.mkdtemp(prefix="sentinel_sandbox_")
        ignore_patterns = shutil.ignore_patterns(
            ".git", ".venv", "__pycache__", "*.pyc", "dist", "node_modules"
        )
        for item in os.listdir(source_dir):
            s = os.path.join(source_dir, item)
            d = os.path.join(temp_dir, item)
            if os.path.isdir(s):
                if item not in [".git", ".venv", "node_modules"]:
                    shutil.copytree(s, d, ignore=ignore_patterns)
            else:
                shutil.copy2(s, d)
        return temp_dir

    async def execute_command(self, cmd: list[str], cwd: str, timeout: int | None = None) -> dict:
        timeout = timeout or self.default_timeout
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            cwd=cwd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE
        )
        self._active_pids.add(proc.pid)
        try:
            stdout, stderr = await asyncio.wait_for(
                proc.communicate(),
                timeout=timeout
            )
            return {
                "exit_code": proc.returncode,
                "stdout": stdout.decode("utf-8", errors="replace"),
                "stderr": stderr.decode("utf-8", errors="replace"),
                "timed_out": False
            }
        except asyncio.TimeoutError:
            self._kill_process_tree(proc.pid)
            return {
                "exit_code": -1,
                "stdout": "",
                "stderr": f"Execution timed out after {timeout} seconds",
                "timed_out": True
            }
        finally:
            self._active_pids.discard(proc.pid)

    def _kill_process_tree(self, pid: int):
        try:
            parent = psutil.Process(pid)
            for child in parent.children(recursive=True):
                try:
                    child.kill()
                except psutil.NoSuchProcess:
                    pass
            parent.kill()
```

```
        except psutil.NoSuchProcess:
            pass
```

D.2 Abstract Syntax Tree Mutation Finder (`sandbox/mutator.py`)

```
import ast

class ASTMutationFinder(ast.NodeVisitor):
    def __init__(self):
        self.mutations = []

    def visit_Compare(self, node):
        op_map = {
            ast.Eq: ast.NotEq,
            ast.NotEq: ast.Eq,
            ast.Lt: ast.GtE,
            ast.LtE: ast.Gt,
            ast.Gt: ast.LtE,
            ast.GtE: ast.Lt,
        }
        for idx, op in enumerate(node.ops):
            if type(op) in op_map:
                self.mutations.append({
                    "line": node.lineno,
                    "type": "COMPARE_OP",
                    "original": type(op).__name__,
                    "replacement": op_map[type(op)].__name__,
                    "target_node": node,
                    "op_idx": idx,
                    "new_op_class": op_map[type(op)]
                })
        self.generic_visit(node)

    def visit_BinOp(self, node):
        bin_map = {
            ast.Add: ast.Sub,
            ast.Sub: ast.Add,
            ast.Mult: ast.Div,
        }
        if type(node.op) in bin_map:
            self.mutations.append({
                "line": node.lineno,
                "type": "ARITHMETIC_OP",
                "original": type(node.op).__name__,
                "replacement": bin_map[type(node.op)].__name__,
                "target_node": node,
                "new_op_class": bin_map[type(node.op)]
            })
        self.generic_visit(node)

    def visit_Constant(self, node):
        if isinstance(node.value, bool):
            self.mutations.append({
                "line": node.lineno,
                "type": "BOOLEAN_FLIP",
                "original": str(node.value),
                "replacement": str(not node.value),
                "target_node": node,
                "new_value": not node.value
            })
        self.generic_visit(node)
```

APPENDIX E: SYSTEM VERIFICATION TEST CATALOGUE

Table E.1: Detailed Verification Test Suite Execution Matrix

Test Case ID	Feature / Component Under Test	Precondition / Input Vector	Expected Behavioral Result	Observed Outcome	Verification Audit Status
TC-01	API Server Health Endpoint	Server active; DB initialized	Returns HTTP `200` with `status: healthy`	HTTP `200` with connected DB	**VERIFIED (Live Audit)**
TC-02	Database SQLite WAL Mode	DB engine initialized	WAL journal mode active; foreign keys enabled	Verified via PRAGMA queries	**VERIFIED (Live Audit)**
TC-03	Startup Crash Recovery	Runs with status `running` in DB	Server transitions runs to `interrupted` at boot	All interrupted runs updated	**VERIFIED (Code Audit)**
TC-04	AST Cyclomatic Complexity	Source file with 4 `if` branches	Computes complexity $V = 5$	Visitor increments correctly	**VERIFIED (Code Audit)**
TC-05	Security Smell Detection	SQL query using string formatting	Flags `SQL_INJECTION` with CRITICAL severity	Accurately flagged	**VERIFIED (Live Demo)**
TC-06	Pytest Test Generation	Valid Python function signature	Generates runnable `pytest` test function	Modular test suite written	**VERIFIED (Live Demo)**
TC-07	Subprocess Execution Timeout	Long-running subprocess (> 60s)	Process-tree terminated; flags timeout	Process terminated via psutil	**VERIFIED (Code Audit)**
TC-08	Flakiness Detection Engine	Test passing 2 of 3 retries	Categorizes status as `flaky` with penalty	Flags flaky test status	**VERIFIED (Code Audit)**
TC-09	Defect Deduplication (MD5)	Identical defect reported across runs	Identical MD5 hash generated; prevents dups	Single record maintained	**VERIFIED (Code Audit)**
TC-10	Pre-Flight Patch Verification	Candidate patch with syntax error	Regression tests fail; patch rejected	`patch_verified = False`	**VERIFIED (Code Audit)**
TC-11	Zero-Division Patch Verification	`billing.py` zero-division defect	Guard inserted; tests pass; patch verified	`patch_verified = True`	**VERIFIED (Live Demo)**
TC-12	Dual-Agent Debate Review	Verified bug submitted to critic	Debate logged; consensus score computed	Consensus score $\ge 95%$	**VERIFIED (Live Demo)**
TC-13	AST Operator Mutation	Comparison operator in source file	Mutant generated, executed, and killed	Mutant status: `killed`	**VERIFIED (Live Demo)**
TC-14	Zip-Slip Upload Protection	ZIP with `../` relative path	Rejects extraction; raises security error	Exception raised; aborts	**VERIFIED (Code Audit)**
TC-15	WebSocket Event Streaming	Client connected to `/ws/runs/{id}`	Broadcasts state transitions in real time	Events streamed cleanly	**VERIFIED (Live Audit)**

APPENDIX F: SETUP AND EXECUTION MANUAL

F.1 Prerequisites

- **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS 12+
- **Python Version:** Python 3.11 or Python 3.12 (Standard CPython)
- **Node.js Environment:** Node.js ≥ 18.0 and npm (optional, only if rebuilding the frontend from source)

F.2 Virtual Environment Setup and Installation

1. Clone or extract the project repository to your preferred local directory:

```
cd "S5 Mini Project"
```

2. Create and activate a Python virtual environment:

```
# On Windows:
python -m venv .venv
.venv\Scripts\activate

# On Linux / macOS:
python3 -m venv .venv
source .venv/bin/activate
```

3. Install the required Python dependencies:

```
pip install -r requirements.txt
```

F.3 Environment Configuration

Copy the sample environment file to create your local `.env` configuration:

```
# On Windows:
copy .env.example .env

# On Linux / macOS:
cp .env.example .env
```

Note: The default configuration uses the `OfflineLLMProvider`, which requires **no API keys**. Cloud providers (Google Gemini or OpenAI) can be enabled optionally by adding their respective API keys to the `.env` file.

F.4 Starting the Application

The project includes automated startup scripts for Windows and Unix systems:

```
# On Windows:
run.bat

# On Linux / macOS:
chmod +x run.sh
./run.sh
```

Alternatively, you can launch the backend server directly using Uvicorn:

```
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

F.5 Accessing the Web Dashboard

Once the server is running, open your web browser and navigate to:

```
http://127.0.0.1:8000
```

- The FastAPI server serves the pre-compiled React dashboard directly from `frontend/dist/`.
- To launch the demo analysis, click **"Launch Demo"** on the dashboard. This registers the reference `demo/ecommerce/` codebase and executes the full nine-stage analysis pipeline.

F.6 Running the Automated Test Suite

To execute the automated unit and integration tests:

```
pytest tests/ -v
```

To run tests with statement coverage reporting:

```
pytest --cov=agents --cov=backend --cov=sandbox tests/
```